

1) What is **metadata**?

ANS - : Metadata stores table structure

👉 It does **not create tables**, it just **remembers their structure**.

2) **bind=engine**

ANS - : "**bind=engine** tells SQLAlchemy which database connection to use when creating the tables."

👉 **engine** तुम खुद बनाते हो, ये SQLAlchemy का object होता है जो database से connection बनाता है।

3) What does **create_all()** do?

ANS - : **create_all()** uses it to create tables in the database.

4) what is the diffrent between put and patch request

Ans :- PUT is used to update the entire resource, while PATCH is used to update only specific fields without affecting the rest.

5) Path Parameter (URL ke andar)

1. Path Parameter ka Code

Path parameter **URL mein bhi likhna padta hai aur function mein bhi**.

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id}
```

Dhyan do

Decorator:

```
@app.get("/users/{user_id}")
```

Yaha **{user_id}** likha hai.

Aur function mein bhi wahi naam:

```
def get_user(user_id: int):
```

⚠️ Dono ka **name same hona chahiye**.

Request:

GET /users/10

Output:

```
{  
  "user_id": 10  
}
```

Ans -: FastAPI supports path, query, body, header, and cookie parameters, each used for different types of data in a request.

Path Parameter (URL ke andar)

```
from fastapi import FastAPI  
  
app = FastAPI()  
  
@app.get("/users/{user_id}")  
def get_user(user_id: int):  
    return {"user_id": user_id}
```

/users/5

2. Query Parameter (URL ke baad ?)

👉 Used for filtering / optional data

```
@app.get("/users/")  
def get_users(limit: int = 10, skip: int = 0):  
    return {"limit": limit, "skip": skip}
```

Example request:

/users/?limit=5&skip=2

Yaad Rakhne ka Trick 🔥

Path Parameter	Query Parameter
URL mein <code>{}</code> ke andar likhte hain	URL mein <code>{}</code> nahi hota
Function mein same name likhte hain	Function mein normal parameter likhte hain
Example: <code>/users/{user_id}</code>	Example: <code>/users?city=Pune</code>
Mostly required	Required ya optional dono ho sakta hai

4. Header Parameter

👉 Used for tokens / auth

```
from fastapi import Header

@app.get("/items/")
def get_items(token: str = Header()):
    return {"token": token}
```

Example header:

token: abc123

6) can we pass body into the get request

ANS : - Although HTTP allows a body in GET requests, it is not recommended because most clients and frameworks don't support it properly. Instead, we should use query parameters or switch to POST if a request body is needed.

Easy English:

"GET request should not have a body. We use query parameters instead."

Hindi:-+

“GET request mein body use nahi karte, uski jagah query parameters use karte hain.”

7) What is yield keyword?

Ans -: Generators help in optimizing memory usage because they generate values on demand instead of storing them

"Yield is used to create generators in Python.

It returns values one by one instead of storing everything in memory.

The function pauses after every yield and resumes from the same point when called again."

8) what depend method do

ANS -: Interview Answer

“**Depends()** in FastAPI is used for dependency injection. It automatically calls a function and provides its result to the route, helping in code reuse and cleaner architecture.”

“Depends ek function ko automatically call karke uska result route me inject karta hai.”

9) why do we use db.refresh

ANs -: 🎯 Short Answer (what interviewers want)

“In SQLAlchemy, **db.refresh()** is used to reload an object from the database after a commit so that we get updated values like auto-generated IDs or default fields.”

10) What is a dunder method?

ANs -:

```
__init__
__str__
__len__
class Student:
    def __init__(self, name):
        self.name = name

    def __str__(self):
        return f"Student name is {self.name}"

s = Student("Rahul")
print(s)
```

Common dunder methods

- `__init__` → constructor (object banate time run hota hai)
- `__str__` → string representation
- `__len__` → length define karta hai
- `__add__` → + operator ka behavior

Main Difference

Normal String	<code>__str__</code> Method
सिर्फ text data होता है	object को string में convert करने का method
"Hello" जैसी value	class के अंदर define होता है
data store करता है	object का readable output देता है
type = <code>str</code>	special function है

`__str__` क्या होता है?
`__str__` एक special method (magic method) है।
जब भी हम किसी object को `print()` करते हैं, तब Python internally `__str__()` method को call करता है।

Example बिना `__str__`

```
class Student:
    def __init__(self, name):
        self.name = name
```

```
s1 = Student("Rahul")
```

```
print(s1)
```

Output कुछ ऐसा आएगा:

```
<__main__.Student object at 0x0000001>
```

यह useful नहीं है।

3. `__str__` use करने के बाद

```
class Student:
    def __init__(self, name):
        self.name = name

    def __str__(self):
        return f"Student Name: {self.name}"
```

```
s1 = Student("Rahul")
```

```
print(s1)
```

Output:

```
Student Name: Rahul
```

अब output readable हो गया।

Simple samjao

`__init__()` -----value store karna

`__str__` -----value ko print show karna

Dunder methods are special methods in Python that allow custom objects to work with Python's built-in functions and operators. We use them when we want our class to behave like a built-in type."

If the interviewer asks "Why not use normal functions?", say:

"We can use normal methods, but Python's built-in syntax does not call them automatically. Dunder methods are special hooks that Python recognizes.

Normal methods are for your own functionality, while dunder methods tell Python how your objects should behave with built-in operations such as `+`,

len(), print(), comparisons, iteration, and object creation.

Dunder methods are special methods in Python. They are used to tell Python how a custom object should behave with built-in functions and operators such as `print()`, `len()`, `==`, and `+`. Normal methods contain the logic, but dunder methods allow Python to call that logic automatically."

Interview Answer

"Dunder methods are special methods in Python with double underscores, used to define the behavior of objects, such as initialization, printing, and operator overloading."

11) Why do we use `pass` in a class method?

ANS -: Hindi:

`pass` tab use hota hai jab **abhi method ka code nahi likhna hai, lekin method banana zaroori hai**

👉 Ye ek **empty placeholder** hai

Interview Answer

"`pass` is used as a placeholder when a method or class is required syntactically but no implementation is needed yet."

12) Difference between Class, Object, and Instance

1. Class

Easy English:

A **class is a blueprint** to create objects.

👉 It defines:

- Properties (variables)
- Behavior (methods)

Hindi:

Class ek blueprint (design) hota hai jisse objects bante hain

```
class Student:
    def __init__(self, name):
        self.name = name
```

2. Object / Instance

Easy English:

An **object** is a real thing created from a class

👉 Instance = object (both mostly same)

Hindi:

Object ya instance class ka actual banaya hua version hota hai

```
s1 = Student("Rahul")
s2 = Student("Amit")
```

✅ Concept + Example Together

Concept	Meaning	Example
Class	Blueprint / design	<code>class Student:</code>
Object	Real item created from class	<code>s1 = Student("Rahul")</code>
Instance	Specific object of class	<code>s1, s2</code>

Final Interview Answer (with example)

"A class is a blueprint, and objects (instances) are created from it. For example, **Student** is a class, and **s1 = Student("Rahul")** is an instance of that class.

13) what is variables .

Ans - they are container for future use

14) interviewer asking what is dynamic typing in python.

Ans - Dynamic typing in Python means you don't need to tell Python the type of a variable. Python decides the type when the program runs, based on the value you give. Also, the same variable can hold different types of values at different times.

"Python is a dynamically typed language. This means we don't need to declare the data type of a variable explicitly. The type is determined automatically at runtime based on the value assigned. A variable can also refer to objects of different types during execution."

15)

GET → lo (data lo)

POST → do (naya data do)

PUT → replace karo

PATCH → thoda change karo

DELETE → hata do

16) Class Kaise banate hai

```
Class Car :  
    color = "blue"  
Car1 = Car() -----object  
print(car1.color)
```

__init__ Function Constructor All classes have a function called **__init__()**, which is always executed when the object is being initiated.

The **self** parameter is a reference to the current instance of the class, and is used to access variables that belongs to the class.

"**__init__**" is a special method in Python called a constructor. It is automatically executed whenever a new object of a class is created. We use it to initialize the object's attributes with their initial values."

creating class

```
## parameterizes contratuer
class Student:
    def __init__( self, fullname ):
        self.name = fullname
```

```
#creating object
s1 = Student( "karan" ) ---- constaruter
print( s1.name )

## __init__ (initialize" (shuru me set karna))
```

- **Student** = class
- **s1** = instance (ya object)

✓ Python me **instance** = **object** (dono same hi use hote hain)

```
#deafult constratuer
class Student:
    def __init__(self)
        pass
```

Class & Instance Attributes

```

class Student:
    school_name = "ABC School" # Class Attribute

    def __init__(self, name):
        self.name = name # Instance Attribute

s1 = Student("Rahul")
s2 = Student("Aman")

print(s1.name, s1.school_name)
print(s2.name, s2.school_name)

```

Output:
Rahul ABC School
Aman ABC School

Methods :-

Methods are functions that belong to objects

```

#creating class
class Student:
    def __init__( self, fullname ):
        self.name = fullname
    def hello( self ):
        print( "hello", self.name)

#creating object
s1 = Student( "karan" )
s1.hello( )

```

Static Methods :-

Methods that don't use the self parameter (work at class level)

"@staticmethod" is used for utility methods that do not depend on instance or class variables.

```

class Student:

```

```
@staticmethod
def college( ):
    print( "ABC College" )
```

* **(Decorators)** allow us to wrap another function in order to extend the behaviour of the wrapped function, without permanently modifying it

Important : four pillars OOPS

Abstraction-:

Hiding the implementation details of a class and only showing the essential features to the user.

Relation samajhne ke liye:

- 👉 Dog IS-A Animal
 - 👉 Bird IS-A Animal
 - 👉 Fish IS-A Animal
-

⚡ Ek line me:

- 👉 Jo class inherit hoti hai = Child
- 👉 Jisse inherit karte hain = Parent

@abstractmethod ka kaam:

- Sirf **declare** karna (method ka naam batana)
- Ye sirf bolta hai: "**implement karo warna error milega**"

🧠 Kab kya use kare?

✅ **pass** use karo jab:

- Har subclass ka behavior **different ho**
- Base class ko kuch nahi pata kaise implement karna hai

👉 (Tumhara notification example — perfect case)

✓ Code likho jab:

- Tum default behavior dena chahte ho
- Subclass optionally override kare

Definition:

Abstraction is the process of **hiding implementation details and showing only essential features to the user.**

Link - https://github.com/ritiksontakke/OOPS_Python/tree/main/abstraction

```
from abc import ABC, abstractmethod

class Animal(ABC):
    def __init__(self, name):
        self.name = name

    @abstractmethod
    def speak(self):
        pass

    @abstractmethod
    def move(self):
        pass

    def describe(self):
        print(f"{self.name} says : {self.speak()} and {self.move()}")

class Dog(Animal):
    def speak(self):
        return "woof"

    def move(self):
        return "runs on four legs"
```

```
class Bird(Animal):
    def speak(self):
        return "Tweet"

    def move(self):
        return "files with wings"

animals = [Dog("Rex"), Bird("Ritik")]

for a in animals:
    a.describe()
```

.___ (this is private modifier) class cha andar hidden asto aani pravite variable apan class che aata madhe cha access karu shkto bhayar nahe karu shkt

@property :- # @property kya hota hai?

@property Python me use hota hai jab hum kisi method ko **variable ki tarah access** karna chahte hain.

Difference समझ

Without Property	With Property
<code>s1.percentage()</code>	<code>s1.percentage</code>
function जैसा	variable जैसा

Matlab:

- method likhte ho
- but call variable jaisa karte ho

x() method hai

Par tum `obj.x` likh rahe ho (function call nahi) , yahan () lagana padta

```
class Demo:
    def __init__(self):
        self._x = 10

    @property
    def x(self):
        return self._x

obj = Demo()
print(obj.x)
```

F String -  Jab bhi string ke andar variable ya expression daalna ho →

f-string best hai `print(f"Balance is {acc.get_balance()}")`

f-string (formatted string literal) is a way to insert variables or expressions directly inside a string.

@setter property की value set/update करने के लिए use होता है और इसमें validation logic लगाया जा सकता है।

"**@property** is a built-in decorator in Python that allows us to access a method like an attribute. It is mainly used to implement getters, compute values dynamically, and add validation while keeping the interface clean."

 A **setter** is used to safely update private data in encapsulation.

@setter :- controls validation before updating:

आसान मतलब

चीज

काम

@property value को पढ़ने (get) के लिए

@setter value को बदलने (set) के लिए

पूरा Example

```
class Student:
```

```
    def __init__(self):  
        self._name = ""
```

```
    # Getter
```

```
    @property
```

```
    def name(self):  
        return self._name
```

```
    # Setter
```

```
    @name.setter
```

```
    def name(self, value):  
  
        if len(value) < 3:  
            print("Name too short")  
  
        else:  
            self._name = value
```

```
s1 = Student()
```

```
s1.name = "Ra"
```



```
print(s1.name)
```

यहाँ क्या हुआ?

जब यह चला:

```
s1.name = "Ra"
```

तो directly value store नहीं हुई।

पहले setter चला:

```
@name.setter
```

उसने check किया:

```
if len(value) < 3
```

फिर बोला:

```
Name too short
```

अगर setter नहीं होता

तो गलत value भी save हो जाती।

अब @property क्यों?

ताकि हम value आसानी से पढ़ सकें।

```
print(s1.name)
```

यह internally getter call करता है।

Internally क्या होता है?

जब लिखते हो:

```
s1.name = "Rahul"
```

Python secretly चलाता है:

```
s1.name("Rahul")
```

मतलब setter method।

जब लिखते हो:

```
print(s1.name)
```

Python internally चलाता है:

```
s1.name()
```

मतलब getter method।

Simple Diagram

Set Value ---> @setter ---> validation ---> store

Get Value ---> @property ---> return value

Yes, we can perform validation using normal `if` conditions.

But using `@setter` is a better OOP approach because it provides centralized and automatic validation.

With normal `if` statements, validation logic has to be written manually every time before assigning a value, which can lead to code repetition and mistakes

"Getters and setters are most commonly used with private variables for encapsulation. However, they can also be used for validation, computed properties, logging, and controlling how attributes are accessed or modified."

Getter → a method used to **read/access** a private variable.

"Getters and setters provide controlled access to an object's attributes. A getter is used to retrieve a value, and a setter is used to modify a value while allowing validation or additional logic. In Python, we usually access attributes directly and use `@property` only when we need control over reading or writing the attribute."

Setter → a method used to **update/change** a private variable.

Encapsulation -: Encapsulation in Python (easy English):

Encapsulation is the process of wrapping data and methods into a single unit and restricting direct access to the data.

Key point: It focuses on *data hiding and protection*.

Example (1 line):

Using private variables and accessing them via getters/setters.

👉 In simple words:

It protects the data by allowing access only through methods.

Short Definition (exam-friendly):

Encapsulation is the process of hiding data inside a class and controlling how it is

accessed.

Link GITREPO -:

https://github.com/ritiksontakke/OOPS_Python/tree/main/encapsulation

```
class BankAccount:
    def __init__(self , owner, initial_balance=0):
        self.owner = owner
        self.__balance = initial_balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print(f"Deposit {amount} . New Balance : {self.__balance}")

    def withdraw(self, amount):
        if amount <=0:
            print("Withdraw Amount Must Be Positive")
        elif amount > self.__balance:
            print("Insufficient balance.")
        else:
            self.__balance -= amount
            print(f"Withdraw {amount} . New Balance {self.__balance}")

    def get_balance(self):
        return self.__balance

acc = BankAccount("ritik" , 100)
acc.deposit(500)
acc.withdraw(50)
print(acc.get_balance())

# Using the class
account = BankAccount(1000)

account.deposit(500)
account.withdraw(300)
```

```
print("Balance:", account.get_balance())
```

Output

```
Deposited: 500  
Withdrawn: 300  
Balance: 1200
```

```
👉 Yehi __init__ ka kaam hai  
class Test:  
    def hello(self):  
        print("Hello")  
  
    def __init__(self):  
        print("Init chala")  
t = Test()  
Init chala
```

Tum pooch rahe ho:

Agar `hello()` upar hai aur `__init__()` neeche hai, to pehle kaunsa chalega?

🔥 **Direct answer:**

👉 Hamesha `__init__` hi pehle chalega

👉 Chahe woh upar likha ho ya neeche

Inheritance :- when one class (child/derived) derives the properties & method of another class(parent/base)

Defi - 2

Inheritance in Python is a core concept of **object-oriented programming (OOP)** that allows one class (called the *child* or *derived* class) to **reuse and extend** the properties and methods of

another class (called the *parent* or *base* class).

`@staticmethod`

A static method is a method inside a class that does not use instance or class data. It does not take `self` or `cls`. It is used when we need a function related to the class, but not dependent on object data.”

A static method is placed inside a class because it is logically related to that class, but it does not need access to instance data (`self`) or class data (`cls`). It behaves like a regular function that is grouped inside the class for better organization

```
class MathUtils:

    @staticmethod
    def add(a, b):
        return a + b

# Calling static method without creating an object
result = MathUtils.add(10, 20)

print("Sum:", result)
Output
30
```

n this example:

- `add()` is a **static method** because it uses only the given parameters.
- It does not use `self` (instance data) or `cls` (class data).
- It is called directly using the class name.

A static method is a method inside a class that does not use object data or class data. It does not take `self` or `cls`. We use it for utility functions related to the class.

`__str__` - Without `__str__` → random ID number

With `__str__` → naam + details

`__str__()` — Interview Explanation

✔ Definition:

`__str__()` is a special (dunder) method used to define the string representation of an object.

Super () :- super matlab mai bula raha hu apne parent ke class ko parent class cha sarva function la bolavat ahe mhnun super() use karto aapan

✔ Definition:

`super()` is used to **call methods from the parent (base) class** inside a child class.

`super()` is used in inheritance to access methods and constructors of the parent class. It helps reuse parent class functionality and avoids code duplication.

Important cheez: `super()`

`super().__init__(name, age)`

👉 Yeh parent (Animal) ka constructor call karta hai

👉 Matlab:

- name aur age Animal se aa rahe hain
- `super()` is used in inheritance to access methods and constructors of the parent class. It helps reuse parent class functionality and avoids code duplication.

🔥 Simple diagram

```
Animal
 /  \
Dog   Cat
```

```
class Animal :
    def __init__(self, name , age):
```

```

        self.name = name
        self.age = age

    def eat(self):
        print(f"{self.name} is eating")

    def sleep(self):
        print(f"{self.name} is sleeping")

    def __str__(self):
        return f"Animal : {self.name}, Age : {self.age}"

class Dog(Animal):
    def __init__(self, name, age, breed):
        super().__init__(name, age)
        self.breed = breed

    def fetch(self, item):
        print(f"{self.name} fetched the {item}!")

    def __str__(self):
        return f"Dog : {self.name}, Age: {self.age}, breed : {self.breed}"

class Cat(Animal):
    def __init__(self, name, age, color):
        super().__init__(name, age) # Call parent constructor
        self.color = color

    def purr(self):
        print(f"{self.name} is purring...")

    def __str__(self):
        return f"Cat: {self.name}, Age: {self.age}, Color: {self.color}"

dog = Dog("Bruno", 3, "Labrador")
dog.eat() # Bruno is eating.
dog.fetch("ball") # Bruno fetched the ball!
print(dog) # Dog: Bruno, Age: 3, Breed: Labrador

```

```
print() # Just for spacing
```

OUTPUT :

Bruno is eating

Bruno fetched the ball!

Dog : Bruno, Age: 3, breed : Labrador

Tum puch rahe ho:

Jab **Animal** wale function me already **name** aur **age** hain,
to **Cat** wale function me firse **name** aur **age** kyun likhe?

Bahut acha question hai.

Sabse simple answer

Kyuki jab tum:

```
Cat("Kitty", 2, "White")
```

likhte ho, to values **sabse pehle Cat class** ke paas aati hain — Animal ke paas nahi.

Isliye Cat ko bhi receive karna padega:

```
def __init__(self, name, age, color):
```

fir Cat un values ko Animal ko bhejta hai:

```
super().__init__(name, age)
```

Polymorphism : **Polymorphism** in Python means “**many forms**”—it allows different objects or data types to be treated through the same interface (method or function), but behave differently depending on the object

Method overloading :- same method name, different inputs, different behavior

Overwrite ka simple matlab:

Purani cheez ko hata ke uski jagah nayi cheez aa jaana

Interview Answer (Best Way)

👉 Method Overloading is a type of compile-time polymorphism where multiple methods have the same name but different parameters.

👉 But in Python, true method overloading is not directly supported. Instead, we achieve similar behavior using default arguments or ***args**.

```
def add(a, b):  
    return a + b  
  
def add(a, b, c):  
    return a + b + c  
# Yeh pehle wale function ko overwrite kar dega
```

Overriding matlab kya hota hai?

👉 Method Overriding ka matlab:

Jab **child class**, parent class ke same method ko **apne hisaab se change (replace)** kar deti hai

Interview Answer (Best Way)

👉 Method Overriding is a concept of runtime polymorphism in Python where a child class provides its own implementation of a method that is already defined in the parent class

```
class Animal:
```

```
def sound(self):  
    print("Some sound")  
  
class Dog(Animal):  
    def sound(self):  
        print("Bark")
```

👉 Yaha:

- `Animal` = parent class
- `Dog` = child class
- `sound()` method dono me same naam ka hai

👉 Dog ne parent ka method **override** kar diya

◆ Output samjho

```
obj = Dog()  
obj.sound()
```

Output bark

Quick bolo:

- **Overloading** → same method, different parameters
- **Overriding** → child class changes parent method

Polymorphism code —

```
class Animal :  
  
    def speak(self):  
        print("Some generic animal sound")  
  
# overriding  
  
class Dog(Animal):
```

```

    def speak(self):
        print("woof woof")

class Cat(Animal):
    def speak(self):
        print("meow meow")

class Cow(Animal):
    def speak(self):
        print("Moo!")

class Duck(Animal):
    def speak(self):
        print("Quack!")

# Function using polymorphism
def make_noise(animal):
    animal.speak()

animals = [Dog(), Cat(), Cow(), Duck()]

for a in animals:
    make_noise(a)

```

SUPER METHOD

super() method is used to access method of the parent class

```

class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, name, marks):
        super().__init__(name) # 👉 parent ka code use
        self.marks = marks

```

```
s = Student("Rahul", 90)
```

```
print(s.name)  
print(s.marks)
```

Output

```
Rahul  
90
```

Parent class = basic cheez (jaise Person 🧑)

Child class = uska upgraded version (jaise Student 🎓)

```
class Student:  
    def __init__(self, marks):  
        self._marks = marks  
  
    @property  
    def marks(self):  
        return self._marks  
  
    @marks.setter  
    def marks(self, value):  
        if value >= 0:  
            self._marks = value  
        else:  
            print("Marks negative nahi ho sakte")
```

```
s = Student(80)
```

```
print(s.marks) # get
```

```
s.marks = 90 # set
```

```
print(s.marks)
```

```
s.marks = -10 # invalid
```

@dataclass :-

@dataclass is a decorator used to reduce boilerplate code when creating classes that mainly store data. It automatically generates methods like `__init__`, `__repr__`, and `__eq__`, so we don't have to write them manually."

`__repr__` - object print कैसा दिखे **Representation (object ka representation) print कैसे दिखे**

```
class Student:
    def __init__(self, name):
        self.name = name

    def __repr__(self):
        return self.name
```

```
s = Student("Rahul")
print(s)
```

`__len__` - **`len()`** का result - Length (length of object)
कितनी items हैं

```
class MyList:
    def __init__(self, items):
        self.items = items

    def __len__(self):
        return len(self.items)
```

```
obj = MyList([1, 2, 3])
print(len(obj))
```

Output

3

`__eq__` - `==` comparison - Equal (equality check)

बराबर हैं या नहीं

```
class Student:
    def __init__(self, marks):
        self.marks = marks

    def __eq__(self, other):
        return self.marks == other.marks
```

```
s1 = Student(50)
```

```
s2 = Student(50)
```

```
print(s1 == s2)
```

Output

True

🧠 While Loop vs For Loop (Quick Notes)

♦ 1. For Loop कब use करें?

👉 जब तुम्हें पहले से पता हो कि loop कितनी बार चलेगा

✅ Example:

```
for i in range(5):
    print(i)
```

Use Cases:

- Counting (1 to n)
 - List / String traverse करना
 - Fixed iterations
-

♦ 2. While Loop कब use करें?

- 👉 जब तुम्हें नहीं पता कि loop कितनी बार चलेगा
- 👉 बस condition true होनी चाहिए

✅ Example:

```
i = 0
while i < 5:
    print(i)
    i += 1
```

Use Cases:

- Unknown iterations
 - Condition based problems
 - End से traverse करना
-

Key Differences

Feature	For Loop	While Loop
Control	Fixed	Condition based
Iterations	Known	Unknown
Use	Counting	Logic based

🎯 Simple Rule (Exam Trick)

👉 “कितनी बार?” → **For Loop**

👉 “जब तक?” → **While Loop**

🧩 Example (LeetCode 58)

while i >= 0 and s[i] == ' ':

👉 Spaces कितने हैं पता नहीं → While

while i >= 0 and s[i] != ' ':

👉 Word की length पता नहीं → While

⚡ Short Trick

- For = Range पता है
 - While = Condition पता है
-

🚀 Summary

👉 For loop = Simple & fixed

👉 While loop = Flexible & condition based

⚡ Main Difference (Easy Table)

Feature	Abstraction	Encapsulation
Focus	Important cheeze dikhana	Data ko protect karna
Hides	Implementation details	Internal data

Purpose	Complexity reduce karna	Security & control
Example	Interface / abstract class	Private variables + getters/setters

SQLAlchemy - relation one to many

USER - one Post -Many

```
from sqlalchemy.orm import relationship
from sqlalchemy import Column, Integer, String
```

```
class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True)
    name = Column(String)
```

relation

One to many relation

posts = relationship("Post", back_populates="user")

posts -- variable tu kuch bhi le sakte ho

Post — Yaha "Post"  class/model ka naam hai.

back_populates="user" — ya user apne post model mai kiya hai vaha se aaya

```
user = relationship("User", back_populates="posts")
```

```
User model
posts = relationship("Post", back_populates="user")
Post model
user = relationship("User", back_populates="posts")
```

ASYNCR AND AWAIT

async and **await** in Python are used for asynchronous, non-blocking programming. In SQLAlchemy, async support improves concurrency and scalability by allowing the application to

process other requests while waiting for database operations to complete. It is especially useful in FastAPI and high-concurrency systems."

You can explain it simply like this in an interview:

"**async** and **await** are used for asynchronous programming in Python. They help the program handle slow tasks like API calls or database queries without blocking the entire application."

Simple explanation

- **async** → defines an asynchronous function
- **await** → waits for a task to finish without stopping the whole program

```
Using async and await  
async def get_data():  
    return "Hello"
```

```
result = await get_data()  
print(result)
```

Yahan:

async → function ko asynchronous banata hai.

await → bolta hai "jab tak result aaye, tab tak dusre kaam kar lo".

async and **await** help Python run tasks without stopping the whole program.

While waiting for database data or an API response, the program can do other work.

File Handling in Python — Interview Explanation

What is File Handling in Python?

File handling in Python is used to perform operations on files such as:

- Creating files
- Reading files
- Writing data into files

- Appending data
- Deleting files

Why Do We Use File Handling?

File handling is used to store data permanently.

Without files:

- Data stays only in RAM
- Data is lost after program execution

With files:

- Data can be saved permanently
- Data can be reused later

. File Modes Samjho

Mode	Meaning
------	---------

"r"	Read file
-----	-----------

"w"	Write file
-----	------------

"a"	Append data
-----	-------------

"x"	Create new file
-----	-----------------

"rb"	Read binary
------	-------------

"wb" Write binary

Character	Meaning
'r'	open for reading (default)
'w'	open for writing, truncating the file first
'x'	create a new file and open it for writing
'a'	open for writing, appending to the end of the file if it exists
'b'	binary mode
't'	text mode (default)
'+'	open a disk file for updating (reading and writing)

Read file code

```
f = open("demo.txt" , "r")
data = f.read()
print(data)
f.close()
```

Write file – entire file data update

```
f = open("demo.txt" , "w")
f.write("I Am Ritik")
f.close()
```

Old file data replace like old data is ritik so replace with i am ritik

Append file — append madhe jo old data tey last madhe data add hoto

```
f = open("demo.txt" , "a")
f.write("sontakke")
f.close()
```

With Syntax

```
With open("demo.txt", "a") as f:
```

```
data = f.read()
```

Delet file

```
Import os  
  
os.remove("sample.txt")
```

1. File Kya Hoti Hai?

`.txt` ek simple text file hoti hai.

Example:

hello.txt

Isme normal text store hota hai.

2. File Open Karna

Python me file open karne ke liye:

```
open("filename", "mode")
```

Example:

```
file = open("hello.txt", "w")
```

3. File Modes Samjho

Mode	Meaning
"r"	Read file
"w"	Write file
"a"	Append data
"x"	Create new file
"rb"	Read binary
"wb"	Write binary

4. /create-file

New file create karna:

```
file = open("myfile.txt", "w")
```

```
file.close()
```

Kya hua?

- "w" mode new file create karta hai
 - `close()` file band karta hai
-

Better Way (Recommended)

with open("myfile.txt", "w") as file:

pass

`with` automatically file close kar deta hai.

5. /write-file 🖋️

File me text likhna:

```
with open("myfile.txt", "w") as file:  
    file.write("Hello World")
```

Output in file:

Hello World

Multiple Lines Write Karna

```
with open("myfile.txt", "w") as file:  
    file.write("Line 1\n")  
    file.write("Line 2\n")
```

6. /read-file 📖

File ka data read karna:

```
with open("myfile.txt", "r") as file:  
    data = file.read()
```

```
print(data)
```

Output

Line 1

Line 2

7. Line By Line Read

```
with open("myfile.txt", "r") as file:  
    for line in file:  
        print(line)
```

8. /append-file +

Purane data ko delete kiye bina new data add karna:

```
with open("myfile.txt", "a") as file:  
    file.write("\nNew Line Added")
```

9. File Exists Ya Nahi Check Karna

```
import os  
  
if os.path.exists("myfile.txt"):  
    print("File exists")  
else:  
    print("File not found")
```

10. File Delete Karna

```
import os  
  
os.remove("myfile.txt")
```

11. Complete Example

```
# create + write
```



```
with open("student.txt", "w") as file:  
    file.write("Name: Rahul\n")  
    file.write("Age: 20")
```

```
# read  
with open("student.txt", "r") as file:  
    data = file.read()  
  
print(data)
```

12. /download-file

Agar aap Python web frameworks use karte ho (Flask/Django), tab file user ko download kara sakte ho.

Flask Example

```
from flask import Flask, send_file  
  
app = Flask(__name__)  
  
@app.route("/download")  
def download():  
    return send_file("myfile.txt", as_attachment=True)  
  
app.run()
```

Important Interview Question

Difference Between **w** and **a**

w

Old data delete karta

a

Old data preserve karta

hai

hai

Fresh write

Append write

Binary read mode (**rb**) is used to read files in binary format instead of text format. It is mainly used for non-text files such as images, PDFs, audio, and videos. In binary mode, data is returned as bytes."

Multithreading –

Multithreading in Python is used to execute multiple tasks concurrently within a single process, mainly for improving performance in I/O-bound applications

GIL (Global Interpreter Lock) is a mutex used in CPython that allows only one thread to execute Python bytecode at a time, making memory management thread-safe but limiting true parallel execution of CPU-bound threads.

Use Multithreading When:

जब:

- existing blocking libraries use करनी हों
- small concurrency चाहिए
- background tasks चाहिए

Examples:

- file uploads
- GUI apps
- lightweight parallel tasks

Use Multiprocessing When:

CPU-heavy work हो।

Examples:

- ML
- image processing

Core Difference

Multithreading	Multiprocessing
One process	Multiple processes
One GIL	Multiple GILs
Shared memory	Separate memory
No true CPU parallelism	True parallelism
Lightweight	Heav

The GIL is a mutex in CPython that allows only one thread to execute Python bytecode at a time, mainly to make memory management thread-safe and prevent race conditions.

Easy Memory Trick

Threading

👉 “Mujhe extra workers chahiye.”

```
import threading
```

```

import time

def func(seconds):
    print(f"sleeping for {seconds} second")

    time.sleep(seconds)

time1 = time.perf_counter()
# normal funtion
# func(4)
# func(3)

# threding funtion

t1 = threading.Thread(target=func, args=[4])
t2 = threading.Thread(target=func, args=[3])

t1.start()
t2.start()

t1.join() - mai rukhna chehaita hu
t2.join() - main thread wait करेगा जब तक t1 खत्म नहीं हो जाता।

time2 = time.perf_counter()

print(time2 - time1)

```

1. import threading
2. function बनाओ
3. Thread object बनाओ
4. start()
5. join()

If Interviewer Asks: Why Not Threading?

You can say:

“Threading in Python is limited by the GIL for CPU-bound tasks, so multiple threads cannot

execute Python bytecode truly in parallel. Multiprocessing creates separate processes with separate Python interpreters, allowing true parallel execution.”

If Interviewer Asks: Difference Between `map()` and `submit()`

You can say:

- `map()` is used for applying one function to multiple inputs.
- `submit()` is used for submitting individual tasks asynchronously and gives back a `Future` object.

LOCK :- Lock Kya Karta Hai?

import threading

```
counter = 0
lock = threading.Lock()

def increment():
    global counter

    for i in range(100000):
        with lock:
            counter += 1

t1 = threading.Thread(target=increment)
t2 = threading.Thread(target=increment)

t1.start()
t2.start()

t1.join()
t2.join()
```

```
print(counter)
```

Lock ka use **race condition** avoid karne ke liye hota hai.

Tumhare example me dono threads ek hi variable **counter** ko modify kar rahe hain. Agar lock na ho, to dono threads ek saath **counter += 1** execute kar sakte hain aur value galat ho sakti hai.

Problem Without Lock

Ye line:

```
counter += 1
```

actually internally 3 steps me hoti hai:

```
temp = counter
temp = temp + 1
counter = temp
```

Agar 2 threads same time pe ye karein:

Thread 1	Thread 2
counter = 5	counter = 5
read	read
+1 => 6	+1 => 6
write 6	write 6

Expected tha **7**, but mila **6**.

Isko bolte hain **Race Condition**.

Lock Kya Karta Hai?

Lock ek gatekeeper hai.

```
lock = threading.Lock()
```

Jab koi thread:

with lock:

andar jata hai:

- lock acquire hota hai
- dusre threads wait karte hain
- kaam complete hone ke baad lock release hota hai

Matlab ek time pe sirf 1 thread critical code execute karega.

What is a Lock in Python Threading?

A **Lock** is used to make sure that only one thread accesses a shared resource at a time.

It helps prevent **race conditions** when multiple threads try to modify the same data simultaneously.

“**Multi processing**” in Python usually means **multiprocessing** — running multiple processes at the same time to make programs faster or use multiple CPU cores.

Python has a built-in module called Python **multiprocessing**.

Sabse Important

Python me:

GIL

ki wajah se multithreading CPU-heavy tasks me weak hota hai |

Isliye:

CPU-bound work ke liye multiprocessing use karte hain |

Multiprocessing is used to achieve true parallel execution by running multiple independent processes, mainly for CPU-bound workloads where multithreading is limited by Python's GIL.

Why use multiprocessing?

Normally, Python programs run in a **single process** and often on a single CPU core.

```
from multiprocessing import Process
import os

def worker():
    print("Running in process:", os.getpid())

if __name__ == "__main__":
    p1 = Process(target=worker)
    p2 = Process(target=worker)

    p1.start()
    p2.start()

    p1.join()
    p2.join()

    print("Finished")
```


Quick Comparison

Feature	Multithreading	Multiprocessing	
Runs using	Threads	Processes	
Memory	Shared memory	Separate memory	
CPU cores used	Usually one core for Python code	Multiple CPU cores	
Speed for CPU tasks	Not much improvement	Much faster	
Best for	I/O-bound tasks	CPU-bound tasks	
Communication	Easy (shared variables)	Harder (IPC needed)	
Memory usage	Low	Higher	
Affected by GIL	Yes	No	

Main Difference

Async/Await	Multithreading
Single thread	Multiple threads
Event loop	OS threads
Cooperative	Preemptive
Lightweight	Heavier
No GIL issue usually	GIL affected
Best for huge I/O	Good for I/O too

Authentication

Authentication is the process of verifying the identity of a user or system.

It answers the question:

“Who are you?”

Authorization

Authorization is the process of determining what an authenticated user is allowed to access or perform.

It answers the question:

“What are you allowed to do?”

Password hashing kya hota hai aur kyu use karte ho?

Password hashing security ke liye use karte hain.

Hum password ko plain text me database me store nahi karte.

Signup time par password ko `hash_password()` se hash karke store karte hain.

Login time par `verify_password()` entered password ko stored hashed password se compare karta hai.

```
Isse agar database leak bhi ho jaye to real passwords visible nahi hote.
```

```
hashed_password = hash_password("123456")
```

```
verify_password(  
    "123456",  
    hashed_password  
)
```

Simple English Interview Explanation

“We use password hashing for security. Passwords are not stored in plain text inside the database. During signup, the password is hashed before storing it. During login, the entered password is verified against the stored hashed password using

`verify_password()`. This protects user passwords even if the database is compromised.”

JWT Token as three part

header is a contain the algo and type

Payload contain data and expression

Signature prove the token create a secret key only for server can create valid token

JWT is a secure token-based authentication method where the server generates a token after login, and the client uses that token to access protected APIs.

Signup



Hash Password



Store User

Login



Verify Password



Generate JWT Token

Protected Request



Decode Token



Get Current User



Authorization Check



Allow / Reject

First, the user signs up and the password is hashed before storing in the database. During login, the password is verified and a JWT token is generated. For protected APIs, the token is decoded to get the current user. Then authorization checks compare resource ownership using user_id to allow or deny access.

Key Difference

Authentication

Verifies identity

“Who are you?”

Happens first

Authorization

Verifies permissions

“What can you access?”

Happens after
authentication

SECRET_KEY = "secret"

ALGORITHM = "HS256"

- SECRET_KEY
→ token sign karne ke liye
- HS256
→ encryption/signing algorithm

create_access_token()

```
jwt.encode(...)
```

Ye token create karta hai.

3. oauth2.py

```
oauth2_scheme = OAuth2PasswordBearer(  
    tokenUrl="/users/login"  
)
```

Ye FastAPI ko batata hai:

Token kaha se milega?

Answer:

/users/login

A **function** and a **method** are very similar because both are blocks of reusable code. The main difference is **where they belong and how they are called**.

Feature	Function	Method
Definition	Independent block of code	Function connected to an object/class
Belongs to	No object	An object or class
Call style	Called directly	Called using an object
Example language	C, Python, JavaScript	OOP languages like Python, Java, C++

Function Example (Python)

```
def add(a, b):  
    return a + b  
  
print(add(2, 3))
```

```
## Here, add() is a function because it works independently.
```

Method Example (Python)

```
class Calculator:  
    def add(self, a, b):  
        return a + b
```

```
c = Calculator()  
print(c.add(2, 3))
```

```
## Here, add() is a method because it belongs to the Calculator  
class and is called using object c.
```

“Authentication means verifying the identity of the user, like login with username and password. Authorization means checking what permissions the user has after login, like whether the user can access admin pages or not.”

Session के अंदर mainly ये चीज़ें होती हैं:

- Database connection
- Transaction state
- Query execution context
- Changed objects की tracking

Simple भाषा में:

Session = DB se baat karne wala manager

जब तुम लिखते हो:

```
db = SessionLocal()
```

Session

- DB connection
- transaction
- query handler
- changed data tracker
- commit/rollback manager

Session kya hota hai

Session ko simple language me samjho:

Database se baat karne ka temporary channel.

MIDDLEWEAR :-

Middleware क्या होता है? (Python / Web Development)

Simple Meaning

Middleware एक ऐसा बीच का **layer** (मध्य परत) होता है जो:

- Request को पकड़ता है
- उस पर कुछ काम करता है
- फिर आगे भेजता है
- और Response वापस आने पर उसे भी process कर सकता है

Life Example

मान लो:

 User →  Security Guard →  Office

यहाँ:

- User = Client
- Office = Server
- Security Guard = Middleware

Authentication check

Logging

Error handling

Request filtering

CORS handling

Middleware एक ऐसा software layer होता है जो:

👉 **Client request** और **Server response** के बीच में काम करता है।

इसका काम request को check करना, modify करना, filter करना या process करना होता है।

Important Point

Middleware हमेशा:

✅ Request और Response के बीच काम करता है।

क्योंकि middleware का काम **global level processing** करना होता है, जबकि आपका काम सिर्फ एक specific field (**user_id**) को clean करना है।

Difference समझो

आपका काम

आप सिर्फ ये करना चाहते हो:

#123vicky → vicky

बस।

यह एक simple string operation है।

Middleware क्या करता है?

Middleware हर request पर चलता है।

Login request

Signup request

Profile request

Payment request

Admin request

सब पर।

Problem क्या होगी?

अगर middleware लगा दिया:

```
user_id = re.sub(r"#\d+", "", user_id)
```

तो हर request में unnecessary processing होगी।

छोटा effect होगा, लेकिन architecture wise अच्छा नहीं माना जाता।

Simple Interview Answer

“Middleware is a software layer that works between the client request and the

server response.

It intercepts requests and responses to perform common tasks like authentication, logging, validation, security, and error handling before passing the request to the main application.”

“In my FastAPI project, middleware can be used for global request processing such as logging, request timing, centralized error handling, authentication logging, request tracking, and monitoring API performance Request



Middleware



Router



Service



Repository



Database



Response



Middleware

Q1. Middleware runs before or after request?

✅ Both.

- Before request reaches server
- After response leaves server

Short Technical Definition

“Middleware is a request-response processing layer used to execute code before and after the main application logic.

क्योंकि middleware का काम **global level processing** करना होता है, जबकि आपका काम सिर्फ एक specific field (**user_id**) को clean करना

Hindi + English Mix (Best for Interview)

“Middleware ek intermediate layer hota hai jo client aur server ke beech kaam karta hai.

Ye request aur response ko intercept karta hai aur common tasks perform karta hai like authentication, logging, validation, error handling, etc.”

```
from fastapi import FastAPI
import time

app = FastAPI()

@app.middleware("http")
async def log_request(request, call_next):

    print("Before Request")

    start_time = time.time()

    response = await call_next(request)

    end_time = time.time()

    print("After Response")

    print(
```

```
        "Process Time:",
        end_time - start_time
    )

    return response

@app.get("/")
def home():

    print("Inside API")

    return {
        "message": "Hello"
    }
```

Output

```
Before Request
Inside API
After Response
Process Time: 0.01
```

Browser Response

```
{
  "message": "Hello"
}
```

Difference Table

@app.middleware

app.middleware()

decorator syntax

function registration

simple projects

large projects

inline middleware

external middleware files

shorter

cleaner architecture

Industry Practice

Project Type	Preferred
small	<code>@app.middl e</code>
large	<code>app.middle ware()</code>

asynchronous programming -: In a synchronous API server, every request blocks while waiting for database or network responses. With async frameworks like FastAPI, the server can handle many requests concurrently, improving scalability and reducing resource usage

कहाँ कौन use करें?

Use Async/Await When:

बहुत सारे I/O tasks हों।

Examples:

- APIs
 - chat servers
 - web sockets
 - downloading
 - database calls
-

Example

1000 API calls

Async बहुत efficient होगा।

Libraries:

- FastAPI
- aiohttp

Async

👉 “Mujhe waiting time optimize karna hai.”

Short interview version:

“Async is used to efficiently handle multiple I/O-bound multiple tasks without blocking the program during waiting operations.”

If they ask:

“What is await?”

You can say:

“**await** pauses the current coroutine until the awaited operation completes, while allowing the event loop to run other tasks.”

async aur **await** Python me asynchronous programming ke liye use hote hain — matlab ek task wait kar raha ho (network, API, file, DB), tab CPU idle na baithe, dusre kaam kare

Ye especially useful hota hai:

- APIs
- Web scraping
- Chat apps
- Fast web servers
- Background jobs

async def

Agar function asynchronous banana ho:

```
async def hello():  
    print("Hello")
```

Ye normal function nahi hota.

Ye coroutine hota hai.

async def → function ko asynchronous banata hai

hello() → function call karne par code turant run nahi hota

Ye ek **coroutine** return karta hai

```
print(hello())
```

likhoge, to output kuch aisa aayega:

```
<coroutine object hello at 0x000001...>
```

Kyuki function execute nahi hua, sirf coroutine object bana.

Actual "**Hello**" print karne ke liye coroutine ko run karna padta hai using **await** ya

`asyncio.run()`

```
import asyncio

async def hello():
    print("Hello")

asyncio.run(hello())
```

Output:

Hello

Flow simple words me:

1. `async def` → coroutine banata hai
2. `hello()` → coroutine object deta hai
3. `asyncio.run()` → coroutine ko execute karta hai
4. Fir `"Hello"` print hota hai

Coroutine kya hota hai?

Coroutine ek special function hai jo:

- pause ho sakta hai
- resume ho sakta hai

Jaise game me save/resume.

Async functions ka use tab hota hai jab:

- API calls
- file downloading
- database queries
- waiting tasks

Event Loop :-

Event Loop ko samajhna async programming ka sabse important part hai.

Simple Definition

Event Loop ek manager hai jo async tasks ko handle karta hai.

Ye continuously dekhta rehta hai:

- kaunsa task chal raha hai
- kaunsa wait kar raha hai
- kaunsa ready ho gaya

Aur accordingly tasks ko switch karta rehta hai.

What is an Event Loop in Python Asyncio?

The Event Loop is the core of `asyncio`.

It continuously manages and schedules asynchronous tasks.

Its job is to:

- run tasks
 - pause tasks that are waiting
 - resume tasks when they are ready
-

Simple Explanation

When a task reaches an `await` statement, it pauses itself and gives control back to the Event Loop.

The Event Loop then runs another task instead of wasting time waiting.

Once the waiting operation is complete (like an API response or timer), the Event Loop resumes

the paused task from where it stopped.

```
import asyncio

async def task1():
    print("Task1 Start")
    await asyncio.sleep(2)
    print("Task1 End")

async def task2():
    print("Task2 Start")
    await asyncio.sleep(1)
    print("Task2 End")

async def main():
    await asyncio.gather(task1(), task2())

asyncio.run(main())
```

Output

Task1 Start

Task2 Start

Task2 End

Task1 End

How It Works

1. Event Loop starts `task1`
2. `task1` reaches `await asyncio.sleep(2)`
3. `task1` pauses
4. Event Loop switches to `task2`
5. `task2` completes
6. After 2 seconds, Event Loop resumes `task1`

Important Interview Point

The Event Loop enables concurrency without creating multiple threads for every task.

It efficiently switches between tasks during waiting periods, which makes async programming fast and scalable for I/O operations like

Python में **try-except** (जिसे लोग अक्सर *try-catch* कहते हैं) का इस्तेमाल **errors/exceptions handle** करने के लिए होता है ताकि program अचानक बंद न हो।

Example:

```
try:
    x = 10 / 0
except:
    print("Error aa gaya")
```

Output:

Error aa gaya

क्योंकि 10/0 से ZeroDivisionError आता है।

Specific Exception Handle करना

Best practice है specific exception पकड़ना।

```
try:
    num = int(input("Number: "))
    print(10 / num)

except ZeroDivisionError:
    print("0 se divide nahi kar sakte")

except ValueError:
    print("Invalid input")
```

कैसे काम करता है?

Flow:

1. `try` block चलता है
2. अगर error नहीं आया → `except` skip
3. अगर error आया → matching `except` चलता है
4. program crash नहीं होता

`else` Block

```
else तब चलता है जब error नहीं आता।
try:
    x = int(input())
except ValueError:
    print("Wrong input")
else:
    print("Success:", x)
```

`finally` Block

`finally` हमेशा चलता है — चाहे error आए या नहीं।

```
try:
    file = open("test.txt")

except FileNotFoundError:
    print("File nahi mili")

finally:
    print("Program finished")
```

Best Practices

- ✓ Specific exceptions use करो
- ✓ Important code को try में रखो
- ✓ Cleanup के लिए finally use करो
- ✓ Meaningful error messages दो

Quick Summary

Keyword	Purpose
<code>try</code>	risky code
<code>except</code>	error handle
<code>else</code>	no error
<code>finally</code>	always execute
<code>raise</code>	manually exception throw

Visual Flow:

try

| — no error —> else

| — error —> except

finally always runs

except क्या करता है?

जब `try` block में error आता है:

- Python program रोकने की बजाय
- `except` block में चला जाता है
- वहाँ हम message दिखा सकते हैं या error handle कर सकते हैं

Main Difference

<code>yield</code>	<code>await</code>
Generator में use होता है	Async function में
Values produce करता है	Async task complete होने का wait
Data iteration	Concurrency / async programming
<code>next()</code> से चलता है	event loop से चलता है

yield क्या करता है?

`yield` → value देता है और function pause कर देता है।

await क्या करता है?

`await` → किसी async operation के complete होने तक wait करता है।

Async/await and multithreading both provide concurrency, but async uses a single-threaded event loop with cooperative task switching, while multithreading uses multiple OS threads managed by the scheduler. Async is ideal for high-scale I/O-bound operations, whereas multithreading is useful for blocking I/O and background tasks. CPU-bound work is better handled using multiprocessing.

Easy Decision Table

Situation	Best Choice
1000 API calls	Async/Await
File downloading	Async/Await
GUI background task	Multithreading
CPU-heavy calculation	Multiprocessing

Image processing

Multiprocessing

Web server handling many
users

Async

Interview Style Answer

`asyncio.gather()` is used to run multiple coroutines concurrently and wait for all of them to finish. The `*tasks` syntax is used to unpack a list of coroutines into separate arguments because `gather()` expects individual awaitables, not a list.

Super Important

Collect vs Unpack

Syntax	Meaning
<code>*args</code> in parameter	collect into tuple
<code>*args</code> in function call	unpack tuple
<code>**kwargs</code> in parameter	collect into dict
<code>**kwargs</code> in function	unpack dict

call

Interview Style Answer

`*args` is used to accept any number of positional arguments as a tuple, and `**kwargs` is used to accept any number of keyword arguments as a dictionary. In decorators, they are used to make the wrapper generic so it can work with any function signature

```
for _ in range(100):
```

```
    print("Hello")
```

Explanation:

“Here, the loop runs 100 times, but we do not need the current iteration value, so `_` is used to indicate that the variable is intentionally ignored.”

You can also mention:

“`_` is just a naming convention; it is not a special keyword in Python

VECTOR_DB -

```
from dotenv import load_dotenv

from qdrant_client import QdrantClient, models

import os

load_dotenv()
```

```
QDRANT_API_KEY = os.getenv("QDRANT_API_KEY")
```

```
QDRANT_URL = os.getenv("QDRANT_URL")
```

```
client = QdrantClient(  
    url=QDRANT_URL,  
    api_key=QDRANT_API_KEY  
)
```

```
collection_name = "my_first_collection"
```

```
client.create_collection(  
    collection_name=collection_name,  
    vectors_config=models.VectorParams(  
        size = 4,  
        distance=models.Distance.COSINE  
    )  
)
```

```
collections = client.get_collection()
```

```
print("Existing Collection:" , collections)
```

e collections ko print karega.

Output something like:

```
Existing Collection:
```

```
collections=['my_first_collection']
```

```
# print(client.get_collection(collection_name))
```

```
Ye specific collection ki details deta hai.
```

Distance Metric

```
distance=models.Distance.COSINE
```

Ye batata hai similarity kaise check hogi.

Cosine similarity:

- vectors ka direction compare karta hai
- AI embeddings me sabse common

Qdrant search karega:

👉 "Kaunsa vector query vector ke most similar hai?"

Bahut important question 🔥

```
size=4
```

ka matlab hai:

👉 Har data ko AI **4 numbers** me represent karega.

Ye numbers hi vector hote hain.

Example:

Maan lo text hai:

"I love cricket"

AI usko convert kar sakta hai:

```
[0.12, 0.45, 0.88, 0.33]
```

Yaha:

4 numbers hain

isliye vector size = 4

Qdrant kya karta hai?

Qdrant vectors store karta hai aur similarity search karta hai.

👉 `get_collections()`

sab collections ki list deta hai.

Simple difference:

Function	Kaam
<code>get_collections()</code>	sab collections ki list
<code>get_collection(name)</code>	ek specific collection ki detail

Analogy 🎯

Database table example:

Tables:

users

products

orders

- `get_collections()` → sab tables dikhao
- `get_collection("users")` → users table ki details dikhao

```
def insertdata():

    points = [
        models.PointStruct(
            id=1,
            vector=[0.1, 0.2, 0.3, 0.4],
            payload={"category" : "demo"}
        )
    ]

    client.upsert(
        collection_name=collection_name,
        points=points
    )

    print("data insert succfully")

create_collection()
insertdata()
```

upsert ka meaning

upsert =

update + insert

Matlab:

- ID nahi hai → insert karo
- ID already hai → update karo

collection_name=collection_name

Kis collection me data bhejna hai.

Example:

collection_name="my_first_collection"

points

```
points=points
```

Jo vectors/data tumne banaya tha wo Qdrant ko bhej raha hai.

```
points = [
```

Yaha ek list ban rahi hai jisme multiple points hain.

First Point

```
models.PointStruct(
```

Ye Qdrant ka ek point/object hai

id

```
id=1
```

Har point ka unique ID.

Database primary key jaisa.

vector

```
vector=[0.1, 0.2, 0.3, 0.4]
```

Ye actual embedding/vector hai.

4 numbers hain because collection ka:

```
size=4
```

tha.

payload

```
payload={"category": "example"}
```

Ye metadata hai.

Extra information store karte hain.

Example:

- category
 - text
 - product name
 - user id
 - document info
-

Second point bhi same hai 📌

id=2

vector=[0.2, 0.3, 0.4, 0.5]

```
payload={"category": "demo"}
```

Conceptually 🎯

Point

└── ID

└── Vector (AI meaning)

└── Payload (extra info)

Real AI app example 📌

```
payload={
```

```
"text": "I love cricket",  
  
"category": "sports"  
  
}
```

Aur vector us text ka embedding hoga.

- payload → filtering ke liye

Ek concise 30-second answer: very very IMP

“Python supports multithreading, but due to the GIL in CPython, only one thread executes Python bytecode at a time. So threads are effective for I/O-bound tasks but not ideal for CPU-bound parallelism. For CPU-intensive work we generally use multiprocessing.”

Vector क्या होता है?

Vector numbers की list होती है जो data का meaning represent करती है।

Example:

[0.12, -0.45, 0.91, 0.33]

AI models text/image/audio को vector में convert करते हैं।

इसे embedding भी कहते हैं।

Payload क्या होता है?

Payload मतलब metadata।

Example:

```
{
```

```
"category": "electronics",
```

```
"price": 199,
```


"rating": 4.7

}

Payload filtering के लिए इस्तेमाल होता है।

जैसे:

- price < 500
- category = books
- rating > 4

Payload Types

Qdrant कई प्रकार के payload support करता है:

Type	Example
Keyword	"electronics"
Integer	120
Float	19.99
Bool	true
Geo	latitude/longitude
Datetime	"2024-01-01"
UUID	unique ids

Filtering Logic

Qdrant में filters ऐसे काम करते हैं:

must = AND

must=[

```
category="books",
```

```
rating > 4
```

```
]
```

दोनों conditions true होनी चाहिए।

should = OR

```
should=[
```

```
category="books",
```

```
category="electronics"
```

```
]
```

कोई एक match होना चाहिए।

must_not = NOT

```
must_not=[
```

```
banned=true
```

```
]
```

यह condition नहीं होनी चाहिए।

1. Payload → extra metadata

Example:

```
{
```

```
"id": 1,
```

```
"vector": [0.2, 0.8, 0.4],
```

```
"payload": {  
  "category": "books",  
  "rating": 4.5  
}
```

यहाँ:

- **id** = unique identifier
 - **vector** = AI embedding
 - **payload** = additional info
-

2. Vector क्या होता है?

Vector numbers की list होती है जो data का meaning represent करती है।

Example:

[0.12, -0.45, 0.91, 0.33]

AI models text/image/audio को vector में convert करते हैं।

इसे embedding भी कहते हैं।

Dense Vector

सबसे common vector type।

Example:

[0.1, 0.2, 0.3, 0.4]

यह semantic meaning capture करता है।

अगर दो sentences similar होंगे तो उनके vectors भी similar होंगे।

Sparse Vector

Sparse vectors में बहुत सारे values 0 होते हैं।

Example:

Dense:

[0,0,0,1,2,0]

Sparse format:

```
{  
  "indices": [3,4],  
  "values": [1.0,2.0]  
}
```

इससे memory बचती है।

Multivector क्या है?

एक point में multiple vectors store करना।

Use case:

- ColBERT models
- token-wise embeddings
- multiple representations

Example:

```
"vector": [  
  [0.1,0.2],  
  [0.3,0.4],  
  [0.5,0.6]  
]
```

Named Vectors

एक ही point में अलग-अलग vector spaces रखना।

Example:

- image embedding
- text embedding
- sparse embedding

```
vector={  
  
  "image": [...],  
  
  "text": [...],  
  
  "text-sparse": {...}  
}
```

1. Fixed-Size Chunking

The Approach: Define a number of tokens per chunk (e.g., 200) with a small overlap buffer to preserve context

Pros:

- Simple to implement
- Consistent chunk sizes

- Predictable processing

Cons:

- Ignores natural language boundaries
- May split mid-sentence or mid-thought
- No semantic awareness

. Sentence-Based Chunking

The Approach: Break documents into sentences using a tokenizer, then group sentences into chunks under a specified word count.

. Paragraph-Based Chunking

The Approach: Split on paragraph breaks, leveraging existing document structure.

सबसे Important Insight

Recursive Chunking

Structure preserve करता है

Semantic Chunking

Meaning preserve करता है

Production Recommendation

Simple Apps

✓ Recursive Chunking

Enterprise RAG

✓ Semantic Chunking

Final Golden Rule

Better chunks

=

Better embeddings

=

Better retrieval

=

Better AI answers

यही modern RAG systems की core foundation है।

Text Chunking Strategy Comparison

Method	Strength	Trade-off	Best For
Fixed-Size	Simple, predictable chunks	Ignores structure, breaks meaning	Raw or unstructured text
Sentence	Preserves complete thoughts	Inconsistent sizes	RAG, Q&A systems
Paragraph	Aligns with semantic units	Large variance in length	Docs, manuals, instructional content
Sliding Window	Maintains full context	Redundant, compute-heavy	Reranking, high-recall retrieval
Recursive	Flexible, handles messy input	Heuristic, sometimes brittle	Scraped web content, mixed sources
Semantic	High-quality, meaning-aware	Slower, resource-intensive	Legal, research, critical QA

def search_and_compare(query, k=3):

- **query** = user क्या search कर रहा है
- **k=3** = top 3 results वापस दो

आसान उदाहरण

मान लो user search करता है:

query = "best laptop"

और database में 100 laptops हैं।

अब program हर laptop को score देगा:

Laptop	Score
--------	-------

Laptop A 95

Laptop B 90

Laptop C 88

Laptop D 70

अगर:

$k = 3$

तो function सिर्फ highest score वाले 3 results देगा:

1. Laptop A
2. Laptop B
3. Laptop C

इसी को कहते हैं Top K Results।

HNSW (Hierarchical Navigable Small World)

HNSW ek graph-based indexing algorithm hai jo vector databases me ultra-fast similarity search ke liye use hota hai.

HNSW is a graph-based ANN(**A**pproximate **N**earest **N**eighbor) indexing algorithm used in vector databases for fast similarity search. It creates hierarchical neighbor connections between vectors, allowing efficient traversal and low-latency retrieval for large-scale AI applications like RAG and semantic search.

```
from qdrant_client.models import HnswConfig
# Example m values
fast_config = HnswConfig(m=8, ef_construct=100,
full_scan_threshold=10000)      # Lower recall, less memory, faster
```

build

```
balanced_config = HnswConfig(m=16, ef_construct=100,  
full_scan_threshold=10000) # Default - good balance
```

```
accurate_config = HnswConfig(m=32, ef_construct=100,  
full_scan_threshold=10000) # Better recall, more memory, slower build
```

Low m (Sparse Graph)

m = 8

- Roads kam hain
- Memory kam lagegi
- City banana fast hoga
- Lekin destination tak best route miss ho sakta hai

Matlab search kabhi-kabhi accurate nahi hogi

High m (Dense Graph)

m = 32

- Bahut saare roads/connections
- Har jagah pahunchne ke multiple paths
- Search jyada accurate
- Relevant vectors milne ke chances high

Lekin:

- RAM jyada lagegi
- Index build slow hoga
- Storage bhi jyada lagega

Comparison Table				
m Value	Graph Type	Memory	Build Speed	Search Accuracy
8	Sparse	Low	Fast	Lower
16	Balanced	Medium	Medium	Good
32	Dense	High	Slow	Better
64	Very Dense	Very High	Very Slow	Excellent

ef_construct

Ye parameter decide karta hai:

"Index build karte time algorithm kitni deeply search karega best neighbors find karne ke liye."

Simple words me:

Jab naya vector graph me add hota hai, tab system uske best neighboring vectors dhoondta hai.

`ef_construct` batata hai:

- kitne candidate nodes ko explore karna hai.

`full_scan_threshold = 10000`

iska simple matlab hai:

Agar search ke time candidate vectors ki count 10,000 se kam hai, to HNSW graph use nahi hoga — direct full scan (brute-force search) hoga.

Real-Life Analogy

Small Notebook (100 pages)

Direct page-by-page dekh lo.

Huge Library (10 lakh books)

Index system use karo.

In Qdrant, `size=384` means:

Each embedding vector will contain 384 numbers (dimensions).

For example:

"I love Ritik"

is converted by the embedding model into:

[0.12, -0.44, 0.91, ...]

with total 384 values.

Qdrant needs this size during collection creation because:

- it prepares memory
- builds vector indexes
- performs similarity calculations

All vectors inside one collection must have the same dimension.

If the model produces 384-dimensional embeddings, then:

`size=384`

must be used.

If dimensions mismatch, Qdrant throws an error like:

expected dim: 384, got 768

because mathematical similarity operations require vectors of equal length.

Short interview answer:

"size defines the number of dimensions in each embedding vector. It must match the embedding model output size."

You can explain it in interview in simple English like this:

size represents the embedding dimension.

For example:

size = 384

means every vector stored in the collection must contain exactly 384 values.

This size must match the embedding model output.

If the embedding model returns:

384 dimensions

then the collection should also be:

size=384

If dimensions are larger or smaller, it will not work.

Example:

- Collection expects 384
- But model sends 768

Then Qdrant throws an error like:

expected dim: 384, got 768

because vector similarity calculations require equal-length vectors.

If we do not mention the dimension (size) while creating the collection, Qdrant cannot create the collection.

Because Qdrant needs to know in advance:

- vector structure
- memory allocation
- index configuration

So **size** is mandatory.

Short interview answer:

"**size** defines the vector dimension. All vectors must have the same dimension as the embedding model output. If dimensions mismatch, Qdrant throws an error. If size is not provided, the collection cannot be created."

Dense vs Sparse Vector

Dense Vector

जिसमें लगभग हर जगह value होती है।

Example:

[0.2, 0.8, 0.4, 0.9]

Sparse Vector

जिसमें ज्यादातर values zero होती हैं।

Example:

[0, 0, 0, 7, 0, 0, 1]

Sparse Vectors कहाँ use होते हैं?

बहुत जगह।

1. Recommendation Systems

Netflix, YouTube, Spotify

- user vs items
- ratings
- likes

Named Sparse Vectors क्यों?

क्योंकि एक point के पास multiple vectors हो सकते हैं:

Type	Purpose
Dense	semantic meaning
Sparse	exact keyword match

यह Hybrid Search में useful है।

Important Rules

indices unique होने चाहिए

गलत:

[1,1,2]

values length same होनी चाहिए

Correct:

indices=[1,2]

values=[0.5,0.7]

Sparse vs Dense

Sparse

Exact matching

Keyword search

Mostly zeros

Inverted Index

Exact search

Dense

Semantic matching

Meaning search

Fully populated

HNSW

Approximate search

Here's a simple **interview-style explanation** you can give:

“A dense vector is a numerical representation where most or all elements contain non-zero values. It is commonly used in machine learning and embeddings to capture semantic meaning efficiently.

For example, a word or sentence can be converted into a dense vector like $[0.2, -0.4, 0.9, 1.1]$, where every number contributes information.

Dense vectors are different from sparse vectors, which contain mostly zeros. Modern AI systems prefer dense vectors because they are compact and capture relationships between data points effectively.”

1. What is a sparse vector?

A sparse vector is a high-dimensional vector in which most values are zero and only a few values are non-zero.

Example:

$[0, 0, 0, 5, 0, 0, 1, 0]$

Sparse vectors are mostly zeros because only a few features are relevant for any single data point out of a very large feature space

vector sirf kuch important features ko represent karta hai, baaki features absent hote hain.

2. What is the difference between dense and sparse vectors?

Dense Vector	Sparse Vector
Most values are non-zero	Most values are zero
More memory usage	Less memory usage
Used in embeddings	Used in keyword/BM25 search

Example dense vector:

[0.2, 0.8, 0.5, 0.1]

Example sparse vector:

[0, 0, 5, 0, 1]

3. Why are sparse vectors useful?

Sparse vectors are useful because they:

- save memory
- improve search efficiency
- store only meaningful information
- work well for recommendation and search systems

4. Where are sparse vectors used?

Sparse vectors are commonly used in:

- search engines
- recommendation systems

- NLP
- BM25 search
- hybrid search
- vector databases like Qdrant

5. How are sparse vectors stored efficiently?

Instead of storing all zeros, sparse vectors are stored as:

(index, value)

pairs.

Example:

[0,0,0,0,1.0,0,2.0]

Stored as:

[(4,1.0), (6,2.0)]

7. Why is dot product good for sparse vectors?

Because only common non-zero dimensions contribute to the final score.

If dimensions contain zeros:

$0 \times \text{anything} = 0$

So unnecessary dimensions are automatically ignored.

8. What happens if two sparse vectors have no common non-zero dimensions?

“Dot product multiplies corresponding elements of two vectors and adds them together. It is commonly used to measure similarity between vectors.”

Their dot product becomes:

similarity=0similarity=0similarity=0

meaning they are not similar.

13. What are the disadvantages of sparse vectors?

Disadvantages:

- limited semantic understanding
- high dimensionality
- may miss contextual meaning

11. What is hybrid search?

Hybrid search combines keyword-based search and semantic vector search to improve accuracy and relevance

Hybrid search combines:

- dense vector search (semantic meaning)
- sparse vector search (keyword matching)

for better search accuracy.

0. How does Qdrant support sparse vectors?

Qdrant supports sparse vectors by storing:

indices = [6,7] `

values = [1.0,2.0]

This enables:

- efficient storage
- fast retrieval
- hybrid search

Sparse Vectors in Qdrant — Easy Explanation

In Qdrant, sparse vectors are stored using `sparse_vectors_config`.

Sparse vectors are different from dense vectors because they mostly contain zeros, so we only store the non-zero values.

Example:

[(6, 1.0), (12, 2.0)]

This means:

- index 6 → value 1.0
- index 12 → value 2.0
- all other indexes are 0

Why no size is needed?

For dense vectors, we must define a fixed size like 384 or 768.

But sparse vectors can have different numbers of non-zero values.

One vector may have 2 non-zero values, another may have 100.

So Qdrant does not require a fixed size.

Distance metric

Sparse vectors always use Dot Product for similarity.

$$A \cdot B = \sum_i A_i B_i \quad \text{A} \cdot \text{B} = \sum_i A_i B_i$$

The system only multiplies matching non-zero indexes, which makes search very fast.

Why do sparse vectors need names?

Sparse vectors are not default in Qdrant.

So we must give them a name during configuration.

Example:

```
sparse_vectors_config = {  
    "text-sparse": SparseVectorParams()  
}
```

Here "text-sparse" is the vector name.

Named vectors advantage

Qdrant allows multiple vectors for the same document.

For example:

- one dense vector
- one sparse vector

for the same data point.

This is useful for Hybrid Search.

Hybrid Search

Hybrid search combines:

- Dense vectors → understand meaning
- Sparse vectors → match exact keywords

So search becomes more accurate.

Hybrid search combines keyword-based retrieval and vector similarity search to improve search relevance. It helps retrieve results using both exact term matching and semantic understanding

Simple Interview Summary

Sparse vectors in Qdrant store only non-zero values.

We do not define vector size because sparse vectors can have different lengths.

Qdrant always uses Dot Product for sparse vector similarity.

Sparse vectors must be named using `sparse_vectors_config`.

Qdrant also supports multiple named vectors, which helps in hybrid search using both dense and sparse vectors together.

Types of Search — Interview Explanation

1. Similarity Search

Similarity search finds the most similar vectors or documents to a query.

It compares vectors using similarity metrics like:

- Cosine Similarity
- Dot Product
- Euclidean Distance

Example:

If the query is:

"best smartphone for photography"

the system may return documents about:

- iPhone camera
- Samsung camera phones

even if the exact words are different.

Used in:

- Semantic search
 - Recommendation systems
 - Chatbots
 - RAG applications
-

2. Keyword Search

Keyword search finds documents containing the exact words from the query.

Example:

"iPhone 15 Pro"

It searches exact terms.

Used in:

- Traditional search engines
 - Product search
 - Log search
-

3. Dense Vector Search

Dense vector search focuses on semantic meaning.

Dense embeddings are generated by deep learning models.

Example:

Query:

"car"

Results may include:

- automobile
- vehicle

because meaning is similar.

Dense search is usually approximate for speed.

4. Sparse Vector Search

Sparse vector search compares only matching non-zero features.

It is useful for exact feature matching and keyword-style retrieval.

Qdrant always uses Dot Product for sparse vectors:

$$A \cdot B = \sum_i A_i B_i \quad \text{A} \cdot \text{B} = \sum_i A_i B_i$$

Used in:

- BM25-style retrieval
 - Keyword matching
 - Exact token matching
-

5. Hybrid Search

Hybrid search combines dense and sparse search together.

Dense vectors provide semantic understanding.

Sparse vectors provide exact keyword matching.

This improves search quality.

Example:

Query:

"cheap gaming laptop"

Hybrid search understands:

- "gaming notebook" semantic meaning
 - exact keyword "cheap"
-

6. Exact Search

Exact search compares all relevant vectors directly.

Advantages:

- Very accurate

Disadvantages:

- Slower on huge datasets

Sparse vector search is usually exact.

“HNSW stands for Hierarchical Navigable Small World.

It is an Approximate Nearest Neighbor (ANN) search algorithm used in vector databases for fast similarity search.

HNSW organizes vectors in a multi-layer graph structure where each vector is connected to its nearest neighbors. Upper layers contain long-range connections for fast navigation, while lower layers provide detailed local search.

When a query vector comes in, HNSW quickly traverses the graph to find the most similar vectors instead of comparing against every vector brute-force.

It is widely used because it provides:

- very low latency,
- high recall/accuracy,
- scalability for millions or billions of vectors.

That’s why modern AI systems like semantic search, recommendation systems, and RAG pipelines commonly use HNSW in vector databases.”

Interview answer

“HNSW is a graph-based Approximate Nearest Neighbor algorithm used in vector databases for fast similarity search. It organizes vectors into hierarchical graph layers and efficiently traverses them to find the nearest vectors with low latency and high accuracy

Agar interviewer aur deep pooche:

“Why better than brute force?”

You can say:

“Brute force compares the query with every vector, which becomes very slow at scale. HNSW reduces search complexity using graph traversal, giving near-accurate results much faster.”

“Why hierarchical?”

“Higher layers help quickly jump across distant regions of the vector space, while lower layers refine the search locally. This hierarchy improves search efficiency.”

“Tradeoff?”

“HNSW is approximate, not exact. It uses more memory for graph connections, but gains significant speed improvements.”

Short 30-second version

“HNSW is a graph-based ANN indexing algorithm used in vector databases for fast similarity search. It creates hierarchical neighbor connections between vectors, allowing efficient traversal and low-latency retrieval for large-scale AI applications like RAG and semantic search.”

ann kya hai

ANN ka full form hai **Approximate Nearest Neighbor**.

Ye technique/vector search method hota hai jo large datasets me **similar vectors quickly find** karne ke liye use hota hai.

7. Approximate Nearest Neighbor (ANN) Search

ANN search finds near-best results very quickly.

Instead of checking every vector, it uses optimized structures like HNSW.

Advantages:

- Very fast
- Scalable

Disadvantages:

- May not always return the absolute best match

Used heavily for dense vectors.

8. Filtered Search

Filtered search combines vector search with metadata filters.

Example:

"laptops"

with filters:

- brand = Dell
- price < 50000

Used in e-commerce and recommendation systems.

9. Full Text Search

Full text search is traditional text-based search.

Uses techniques like:

- TF-IDF
- BM25

Search is based on word frequency and text relevance.

Short Interview Summary

There are multiple types of search systems. Similarity search finds semantically similar vectors using metrics like cosine similarity or dot product. Dense search focuses on semantic meaning, while sparse search focuses on exact feature matching. Hybrid search combines both. Exact search gives accurate results, while

ANN search improves speed for large-scale vector databases. Filtered and full-text search are also commonly used in modern search systems.

Feature	BM25	miniCOIL	SPLADE
Exact keyword match	Yes	Yes	Yes
Semantic understanding	No	Medium	High
Speed	Fast	Medium	Slower
Explainability	High	Medium	Low
Expansion	No	Limited	Strong
Best for	Simple search	Smart keyword search	Semantic sparse retrieval

ANN is a fast similarity search technique used to retrieve nearest vectors approximately instead of exact brute-force comparison.

16. कब कौन सा Model Use करें?

BM25 Use करें अगर:

Simple interview explanation

“BM25 is a keyword-based ranking algorithm that scores documents based on how relevant they are to a search query.”

- simple keyword search चाहिए
- fast retrieval चाहिए
- explainability जरूरी है

miniCOIL Use करें अगर:

Ye modern search systems me BM25 ka advanced neural version maana jata hai.

miniCOIL is a lightweight neural sparse retrieval model derived from COIL (Contextualized Inverted List). It generates sparse token-level representations with contextual weighting, enabling efficient semantic retrieval while preserving keyword matching behavior. It is commonly used in hybrid and neural search systems

- exact keyword important हैं
 - context भी चाहिए
 - e-commerce / legal / medical domain है
-

SPLADE Use करें अगर:

SPLADE ek **neural sparse retrieval model** hai jo semantic search aur keyword search ko combine karta hai.

“SPLADE is a neural sparse retrieval model that generates sparse lexical representations using transformer-based contextual embeddings. It expands and weights important terms semantically, enabling efficient retrieval with both keyword matching and semantic understanding.

- semantic recall चाहिए
 - synonym matching चाहिए
 - vocabulary mismatch बड़ा issue है
-

17. Hybrid Search क्या है?

Sparse + Dense दोनों combine करना।

Sparse Search

Strengths:

- exact match
- explainable

- fast

Dense Search

Strengths:

- semantic meaning
 - discovery
 - synonym understanding
-

Hybrid Search Example

Query:

apple slicer

Sparse:

- apple cutter

Dense:

- fruit peeler

दोनों combine करके best results मिलते हैं।

Dense vs Sparse आसान तुलना

Dense Vector	Sparse Vector
Meaning समझता है	Exact शब्द match करता है
Semantic search	Keyword search
Natural language अच्छा handle करता है	Product codes/model numbers में अच्छा
Similar concepts ढूँढ सकता है	Precise matching करता है

Short Summary

Feature	Kaam
Quantization	Memory bachata + fast search
Oversampling	Extra candidates lata
Rescoring	Original vectors se accurate comparison
Reranking	Final best results choose kart

आसान Summary

Situation	Best Option
Real-time updates	upsert
Medium dataset	upload_points
Huge dataset	upload_collection
RAM कम हो	on_disk=True
Fast search चाहिए	Quantization + HNSW RAM में



Scale के हिसाब से क्या चुनें?

Dataset Size	Recommended Method
< 100K	Batch upsert
100K – 1M	upload_points
> 1M	upload_collection

VECTOR DB IMP

In Qdrant, a **Point** is the basic unit of data stored in the database.

Each point has 3 main parts:

1. **Unique ID**

- Used to identify the point uniquely.
- Can be a number or UUID.

2. **Vector**

- A list of numbers called an embedding.
- Represents data like text, images, or audio in mathematical form.
- Used for similarity search.

3. **Payload (Optional)**

- Extra metadata about the point.
- Example: name, category, price, etc.
- Helps in filtering search results.

Why “Dense” Vector?

It is called **dense** because most positions in the vector contain values (not empty or zero).

Example:

[0.21, 0.55, 0.91, 0.44]

Almost every position has useful information.

Dense vectors are numerical embeddings generated by AI models to represent data in high-dimensional space. They capture semantic meaning and are used in vector databases like Qdrant for similarity search.”

Why Dense Vectors Are Used

1. **Semantic Search**

Search by meaning, not exact words.

We use dense vectors to represent data numerically so machines can understand semantic meaning and perform similarity search efficiently.”

SPARSE VECTOR

A **Sparse Vector** is also a vector representation of data, but most values are **0**.

Instead of storing all values, sparse vectors store only the **non-zero values**.

Sparse vector stores only useful values:

[(2, 0.45), (5, 0.78)]

Meaning:

- index 2 → value 0.45
- index 5 → value 0.78

Why Use Sparse Vectors?

Because they:

- save memory
- improve efficiency
- work well for keyword-based search

Dense vs Sparse Vector

Dense Vector	Sparse Vector
Most values are non-zero	Most values are zero
Used for semantic meaning	Used for keyword/token matching
Generated by embeddings	Often generated from text/token frequency
Larger storage	Optimized storage

Real Use Case

Sparse vectors are commonly used in:

- BM25 search
- keyword search
- hybrid search systems

Qdrant can combine:

- Dense vectors → semantic search
- Sparse vectors → keyword search

for better results.

Interview-Friendly Answer

“Sparse vectors are vectors where most values are zero. Instead of storing all values, only non-zero index-value pairs are stored, making storage and search more efficient.”

Multivectors in Simple English

Normally, an AI embedding model generates **one vector for one input**.

Example:

Sentence → One Vector

But some advanced models like ColBERT generate **multiple vectors** for a single sentence or document.

Usually:

- one vector for each word/token

This is called a **Multivector**.

Multivectors store multiple vectors for a single data point. They are used by advanced models like ColBERT, where each token can have its own vector representation for more accurate retrieval.

Dense vs Sparse vs Multivector — When to Use

Type	Use When	Best For
Dense Vector	Need semantic understanding	AI search, chatbot, recommendation
Sparse Vector	Need exact keyword matching	BM25, search engines
Multivector	Need highly accurate ” Ask ChatGPT	Advanced RAG, ColBERT search

Dense vectors are used for semantic search, sparse vectors for exact keyword matching, and multivectors for advanced high-accuracy retrieval systems.

Named Vectors in Simple English

In Qdrant, a single point can store **multiple vectors** instead of just one.

These vectors are called **Named Vectors**.

Each vector has its own name and purpose.

Why Use Named Vectors?

Sometimes one embedding is not enough.

You may want:

- one vector for title
- one vector for description
- one vector for image

```
inside the same point.  
Suppose an e-commerce product has:  
product title  
product description  
product image  
Each can have different embeddings.
```

```
{
  "id": 1,
  "vectors": {
    "title_vector": [0.11, 0.22],
    "description_vector": [0.33, 0.44],
    "image_vector": [0.55, 0.66]
  }
}
```

“Named vectors allow storing multiple embeddings inside a single Qdrant point. Each vector space has its own name and can be searched independently.

Vector Dimensionality

Vector Dimensionality ka matlab hota hai:
ek vector ke andar kitne numbers (features) hain.

Example:

- 384 dimensions → vector me 384 numbers
- 768 dimensions → 768 numbers
- 1536 dimensions → 1536 numbers

AI models text ko numbers me convert karte hain jise **embeddings** ya **vectors** bolte hain.

Ye vectors semantic meaning store karte hain.

Common Models aur Dimensions

Model	Dimensions
OpenAI text-embedding-3-small	1536
OpenAI text-embedding-3-large	3072
Sentence Transformers MiniLM	384
MPNet	768

384-dimensional vector means the embedding contains 384 numerical features.

More dimensions usually capture richer semantic information, but they also increase memory and computation cost.

Why not always use higher dimensions?"

Answer:

“Because higher dimensions improve representation quality, but they also increase latency, storage, and search complexity.

So we choose dimensions based on the tradeoff between accuracy and performance.”

Real-world angle (strong answer)

“For production systems like semantic search or RAG pipelines, dimensions like 768 or 1536 are commonly used because they provide a good balance between retrieval accuracy and efficiency.”

What is FastEmbed?

FastEmbed is an optimized embedding generation library developed by Qdrant.

It is designed for fast, lightweight, and CPU-efficient embedding generation.

Explain semantic search.

Semantic search retrieves results based on meaning rather than exact keywords.

It uses embeddings and vector similarity to find contextually relevant information.

Embedding Comparison

Feature	FastEmbed	Cloud Providers (incl. Qdrant Inference)	Open Source Models
Execution	On-premise (CPU/GPU)	Cloud API	On-premise (CPU/GPU)
Speed	Optimized for low-latency CPU inference	API latency (external) or near-zero (Qdrant)	Varies by model and hardware
Control	High	Low	Maximum (fine-tuning, architecture)
Best for	Qdrant-native, lightweight, fast CPU	Ease of use, managed scaling	Domain-specific customization, full control

Filtering Logic: Building Complex Queries

- **must:** All conditions must be satisfied (AND logic)
- **should:** At least one condition must be satisfied (OR logic)
- **must_not:** None of the conditions should be satisfied (NOT logic)

```
models.Filter(  
    should=[  
        models.Filter(must=[  
            models.FieldCondition(key="category",  
match=models.MatchValue(value="electronics")),  
            models.FieldCondition(key="price",  
range=models.Range(lt=200))  
        ]),  
        models.Filter(must=[  
            models.FieldCondition(key="category",  
match=models.MatchValue(value="books")),  
            models.FieldCondition(key="rating",  
range=models.Range(gte=4.0))  
        ])  
    ]  
)
```

Text Chunking Strategies

1. Fixed-Size Chunking

Fixed-size chunking is a text splitting technique where we divide large text into chunks of a predefined size, such as 200 words or tokens. Sometimes we also keep an overlap between chunks to preserve context continuity.

If chunk size is 200 and overlap is 50:

- Chunk 1 contains words 1–200
- Chunk 2 contains words 151–350

So 50 words are repeated to maintain context.”

this technique is commonly used in RAG (Retrieval-Augmented Generation) systems and vector databases because LLMs cannot process extremely large documents at once.”

Sentence-Based Chunking Explanation

“Sentence-based chunking is a text splitting technique where documents are first divided into sentences using a tokenizer, and then multiple sentences are grouped together until a maximum word limit is reached.”

How It Works

“First, we tokenize the document into sentences using libraries like NLTK. Then we keep adding sentences into a chunk until the word limit is reached.”

Suppose we have this text:

“The HNSW algorithm builds a multi-layer graph. Each node represents a vector in the collection. The algorithm creates shortcuts between layers for faster search.”

Paragraph-Based Chunking Explanation

“Paragraph-based chunking is a technique where text is divided based on paragraph boundaries. Each paragraph becomes a separate chunk, preserving the author’s original structure and related ideas together.”

Short Interview Version (30 Seconds)

“Paragraph-based chunking splits text using paragraph boundaries, keeping related concepts together according to the document’s original structure. It provides strong semantic coherence but can create uneven chunk sizes.”

Difference from Sentence Chunking

Sentence Chunking

Splits by sentences

More controlled size

Better size management

Paragraph Chunking

Splits by paragraphs

Larger semantic blocks

Better topic grouping

Sliding Window Chunking Explanation

“Sliding window chunking is a technique where text is divided into overlapping chunks. Each new chunk shares some words or tokens with the previous chunk to preserve context continuity.”

Why Overlap is Important

“Overlap helps preserve context across chunk boundaries, reducing the chance of losing important information when sentences or concepts span multiple chunks.”

Best Use Cases

“Sliding window chunking is ideal for high-accuracy RAG systems, semantic search, reranking pipelines, and applications where missing information is expensive.”

Difference from Fixed-Size Chunking

Fixed-Size	Sliding Window
No overlap	Uses overlap
Context loss possible	Better continuity
Lower storage cost	Higher storage cost
Simpler retrieval	Better recall

Short Interview Version (30 Seconds)

“Sliding window chunking creates overlapping chunks by moving a fixed-size window across the text with a smaller stride. The overlap preserves context continuity and improves retrieval quality, though it increases storage and processing costs.”

Recursive Chunking Explanation

“Recursive chunking is an advanced text splitting technique where the system tries multiple separators in a hierarchical order until the text fits within the desired chunk size.”

Core Idea

“Instead of splitting text using only one rule, recursive chunking uses fallback strategies.”

It starts with larger separators:

1. Paragraphs (`\n\n`)
2. Lines (`\n`)
3. Sentences (`.`)
4. Words ()
5. Characters

If one chunk is still too large, it recursively moves to the next smaller separator

Why Recursive Chunking is Powerful

“It preserves semantic structure whenever possible while still guaranteeing chunks remain within token limits.”

Short Interview Version (30 Seconds)

“Recursive chunking uses a hierarchy of separators like paragraphs, sentences, words, and characters to intelligently split text. It preserves structure whenever possible while ensuring chunks stay within token limits, making it highly effective for messy real-world documents.”

Semantic-Aware Chunking Explanation

“Semantic-aware chunking is an intelligent text splitting technique where chunks are created based on meaning or topic transitions instead of fixed size or document structure.”

Interview-Level Insight

“Semantic chunking improves retrieval quality because embeddings are stored for semantically coherent concepts instead of arbitrarily split text.”

Short Interview Version (30 Seconds)

“Semantic-aware chunking uses embeddings and similarity scoring to detect topic transitions in text. Instead of relying on structure or token count, it creates chunks based on meaning, resulting in semantically coherent chunks and higher retrieval accuracy.”

Text Chunking Strategy Comparison

Method	Strength	Trade-off	Best For
Fixed-Size	Simple, predictable chunks	Ignores structure, breaks meaning	Raw or unstructured text
Sentence	Preserves complete thoughts	Inconsistent sizes	RAG, Q&A systems
Paragraph	Aligns with semantic units	Large variance in length	Docs, manuals, instructional content
Sliding Window	Maintains full context	Redundant, compute-heavy	Reranking, high-recall retrieval
Recursive	Flexible, handles messy input	Heuristic, sometimes brittle	Scraped web content, mixed sources
Semantic	High-quality, meaning-aware	Slower, resource-intensive	Legal, research, critical QA

HNSW — Hierarchical Navigable Small Worlds.

Parameter Summary

Parameter	क्या Control करता है	बढ़ाने पर क्या होगा
<code>m</code>	connections per node	accuracy ↑ RAM ↑
<code>ef_construct</code>	indexing quality	better graph, slower build
<code>hnsw_ef</code>	search depth	better recall, slower query

Workload के हिसाब से Settings



Fast Search चाहिए

Use:

- low `m`
- low `hnsw_ef`

Best for:

- realtime apps
 - chatbots
-

Maximum Accuracy चाहिए

Use:

- high `m`
- high `ef_construct`
- high `hnsw_ef`

Best for:

- recommendation systems
 - semantic search
-

RAM कम है

Use:

- lower `m`

लेकिन:

- `ef_construct` बहुत कम मत रखना

Important Understanding

Lower `hnsw_ef`

`hnsw_ef` = 32

- very fast
 - lower recall
-

Higher **hnsw_ef**

hnsw_ef = 256

- slower
 - better accuracy
-

Real-Life Analogy

Imagine treasure hunting:

Low **hnsw_ef**

थोड़ी जगह search करो।

Fast 

लेकिन treasure miss हो सकता है 

High **hnsw_ef**

ज्यादा area search करो।

Slow 

लेकिन best result मिलने की probability ज्यादा 

What is HNSW?

HNSW stands for:

Hierarchical Navigable Small Worlds

It is an indexing algorithm used in vector databases like Qdrant to make vector search fast.

Why Do We Need HNSW?

Without indexing, the database would compare the query vector with every vector in the dataset.

That is called:

- brute force search
- very slow for millions of vectors

HNSW solves this problem by creating a smart graph structure.

How HNSW Works

HNSW creates multiple graph layers.

- Top layers → broad connections
- Lower layers → detailed connections

Each vector is connected to similar vectors.

During search:

1. Start from the top layer
2. Move toward closer neighbors
3. Go down layer by layer
4. Finally find the nearest vectors

So the system does not search everywhere.

It searches only in the most relevant area.

1. **m**

Controls how many neighbors each node connects to.

- Higher **m**:
 - better accuracy
 - more memory usage
 - Lower **m**:
 - faster
 - less memory
 - lower accuracy
-

2. ef_construct

Controls index build quality.

- Higher value:
 - better graph quality
 - slower indexing
-

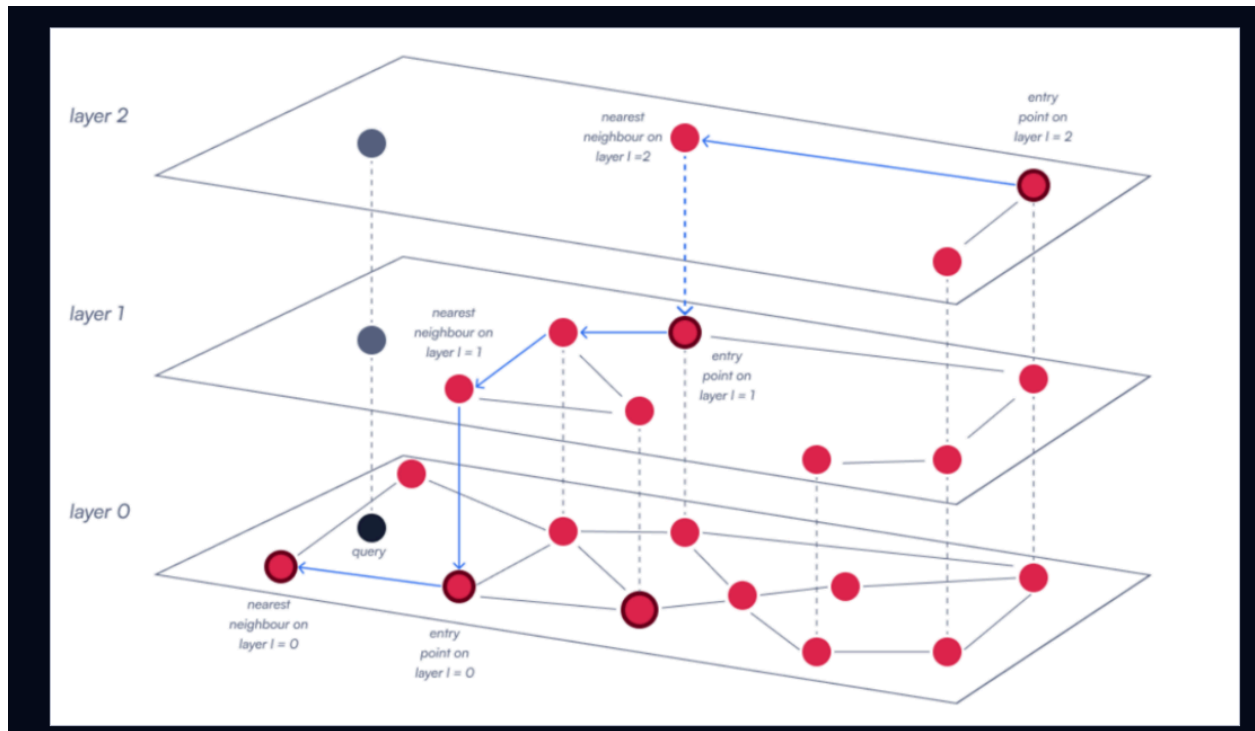
3. hnsw_ef

Controls search quality at query time.

- Higher value:
 - better recall
 - slower search
- Lower value:
 - faster search
 - may miss some results

One-Line Interview Answer

“HNSW is an approximate nearest neighbor indexing algorithm that organizes vectors into a multi-layer graph, allowing fast and scalable similarity search without checking every vector.”



Important Things

● Red dots

Vectors / nodes हैं।

हर node:

- एक embedding vector represent करता है।

● Black dot

यह query vector है।

यानी:

जो चीज search करनी है।

— Lines

Connections between similar vectors |

यानी:

- similar vectors आपस में जुड़े हुए हैं।

Interview में कैसे Explain करना है?

Simple English answer:

“This diagram shows how HNSW performs hierarchical graph search.

It starts from the top layer using a random entry point, moves toward closer neighbors, and gradually descends to lower layers where connections become more detailed.

Instead of checking every vector, it navigates through the graph to quickly find approximate nearest neighbors.”

Interviewer क्या पूछ सकता है?

1. Why multiple layers?

Answer:

Upper layers help fast navigation across the graph, while lower layers provide detailed local search for accuracy.

2. Why is HNSW fast?

Answer:

Because it avoids brute force search and explores only a small part of the graph.

3. What do arrows represent?

Answer:

Graph traversal between neighboring vectors.

4. What is the entry point?

Answer:

Starting node for the search in the top layer.

5. Why is layer 0 largest?

Answer:

The bottom layer contains all vectors and the most detailed connections.

6. Is HNSW exact?

Answer:

No, it is approximate nearest neighbor search, optimized for speed and scalability.

7. Difference between brute force and HNSW?

Answer:

Brute Force	HNSW
Checks every vector	Traverses graph
Slow	Fast

Exact	Approximate
$O(N)$	$\sim O(\log N)$

Kya interviewer diagram draw karne ko bolta hai?

Sometimes YES.

लेकिन usually simple sketch enough होता है।

बस इतना draw कर दो:

Layer 2 -> few nodes

Layer 1 -> more nodes

Layer 0 -> many nodes

Query -> navigate down layers

इतना काफी होता है।

Most Important Interview Line ★

याद कर लो:

“HNSW works like navigating a city using highways first and then local streets to quickly reach the nearest destination.”

यह analogy interviewers को बहुत पसंद आती है।

phle mujhe samjao digram mai kya ho raha q nikale and all

ठीक है 👍

अब interview answer नहीं — पहले diagram itself समझते हैं step-by-step।

यह diagram basically दिखाता है:

Query vector nearest vector तक कैसे पहुँचता है using HNSW graph.

सबसे पहले Image में क्या क्या है?

1. Layers

Image में 3 layers हैं:

Layer 2 (top)

Layer 1

Layer 0 (bottom)

ये layers क्यों हैं?

HNSW full data को multiple levels में divide करता है।

Top Layer

- कम nodes
- long-distance connections
- fast navigation

जैसे:

highways

Bottom Layer

- बहुत सारे nodes
- detailed connections

जैसे:

local roads

2. Red Dots क्या हैं?

हर red dot = एक vector/node

Example:

- image embedding
 - text embedding
 - product embedding
-

3. Black Dot क्या है?

Black dot = Query vector

यानी:

जो चीज search करनी है।

Example:

- “best laptop”
 - “similar image”
 - “semantic search query”
-

4. Lines क्या हैं?

Lines मतलब:

similar vectors आपस में connected हैं।

अगर दो vectors similar हैं:

- graph edge बनता है।
-

अब actual search समझो 🔥

STEP 1 → Search कहाँ से शुरू होता है?

Top layer में right side वाला node:

entry point

से search शुरू होती है।

Question:

क्यों random top node से start करते हैं?

क्योंकि:

- top layer sparse है
- जल्दी navigation possible है

पूरा graph explore नहीं करना पड़ता।

STEP 2 → Closest Neighbor ढूँढना

अब algorithm:

- connected neighbors check करता है
 - कौन query के ज्यादा close है देखता है
-

Diagram में Blue Arrow क्या दिखा रहा?

Blue arrow मतलब:

Current node → Better nearest node

Algorithm धीरे धीरे:

- better vectors की तरफ move करता है।
-

Real Life Analogy

सोचो Google Maps:

- पहले city area चुनता है
 - फिर nearby roads
 - फिर exact street
-

STEP 3 → नीचे वाली layer में जाना

जब top layer में:

- best nearby node मिल जाता है

तब algorithm:

- उसी node से next layer में उतरता है।

Diagram में vertical dotted lines यही दिखाती हैं।

Important Concept ★

हर node हर layer में नहीं होता।

कुछ nodes:

- top layers में भी होते हैं
 - कुछ सिर्फ bottom layer में
-

क्यों?

Top layer:

- shortcuts provide करती है

Bottom layer:

- detailed accuracy देती है
-

STEP 4 → More Detailed Search

अब Layer 1 में:

- ज्यादा nodes हैं
- ज्यादा connections हैं

Algorithm:

- फिर nearest neighbors check करता है
 - query के और close move करता है।
-

STEP 5 → Final Layer (Layer 0)

अब bottom layer में:

- सबसे detailed graph है

Algorithm:

- local search करता है
 - final nearest neighbor find करता है।
-

Bottom Layer Important क्यों है?

क्योंकि:

सारे vectors आखिर में layer 0 में होते हैं।

Diagram का पूरा Flow Simple Language में

Top layer:

Fast rough navigation

Middle layer:

Better refinement

Bottom layer:

Accurate nearest search

पूरा Idea क्या है?

Instead of:

Checking EVERY vector

HNSW करता है:

Jump → refine → refine → nearest result

Diagram क्यों बनाया गया?

यह explain करने के लिए कि:

HNSW:

- brute force नहीं करता
 - graph traversal करता है
 - hierarchical search करता है
 - top-down navigation use करता है
-

Interviewer क्या देखना चाहता है?

कि तुम्हें समझ है:

- ✓ layers क्यों हैं
- ✓ entry point क्या है
- ✓ nearest neighbor traversal क्या है

- ✓ graph search कैसे होती है
 - ✓ brute force क्यों avoid होता है
-

Most Important Understanding ★

Top Layers

Fast direction देते हैं।

Bottom Layers

Accurate answer देते हैं।

One-Line Intuition

“HNSW first quickly reaches the correct neighborhood using upper layers, then uses lower layers to accurately find the nearest vector.”

Interviewer क्या देखना चाहता है? ===== कि तुम्हें समझ है: ✓ layers क्यों हैं ✓ entry point क्या है ✓ nearest neighbor traversal क्या है ✓ graph search कैसे होती है ✓ brute force क्यों avoid होता है

batao sab ke ans

हाँ 👍

ये interviewer के most common HNSW questions हैं।
अब इनके simple easy-English answers याद कर लो।

1. Layers क्यों हैं?

Answer

Layers are used to make search faster.

Upper layers help quickly navigate large areas of the graph, while lower layers provide detailed and accurate search.

Easy Analogy

Upper layer = highways

Lower layer = local streets

पहले highway से सही area पहुँचते हैं, फिर local roads से exact location।

2. Entry Point क्या है?

Answer

Entry point is the starting node from where HNSW begins the search in the top layer.

Why needed?

क्योंकि search कहीं न कहीं से start करनी पड़ती है।

Top layer sparse होती है इसलिए navigation fast होता है।

3. Nearest Neighbor Traversal क्या है?

Answer

The algorithm moves from one node to another node that is closer to the query vector.

Simple Meaning

Current node से:

- nearby connected nodes check करता है
 - जो query के ज्यादा similar हो वहाँ move करता है।
-

Diagram में Blue Arrows क्या हैं?

यही traversal दिखा रहे हैं।

4. Graph Search कैसे होती है?

Answer

HNSW performs graph traversal layer by layer.

It starts from the top layer, finds closer neighbors, and gradually moves to lower layers for more accurate search.

Simple Flow

Top layer → rough search

Middle → better refinement

Bottom → accurate nearest result

5. Brute Force क्यों avoid होता है?

Answer

Brute force checks every vector, which becomes very slow for large datasets.

HNSW avoids this by searching only a small relevant part of the graph.

Complexity

Brute force:

$O(N)O(N)O(N)$

HNSW:

$O(\log N)O(\log N)O(\log N)$

Most Important Interview Answer ★

अगर interviewer बोले:

“Explain HNSW simply”

तो बोलो:

“HNSW is a hierarchical graph-based indexing algorithm used for fast approximate nearest neighbor search. It starts from upper sparse layers for fast navigation and moves down to detailed lower layers to find the nearest vectors efficiently without checking every vector.”

One Super Easy Human Analogy ★

“It works like Google Maps.

First it uses highways to reach the correct area quickly, then local roads to reach the exact destination.”

Combining Vector Search and Filtering

Problem kya hai?

Qdrant similarity search ke liye **HNSW graph** use karta hai.

HNSW ek network/graph जैसा होता है जहाँ हर point दूसरे points से connected होता है.

Isse search bahut fast ho jati hai.

HNSW graph ke bahut saare nodes filter ki wajah se remove ho jaate hain.

To graph ka connection toot sakta hai.

Short Interview Answer (2–3 min)

“Qdrant uses HNSW graphs for fast approximate nearest neighbor vector search.

But in production systems we usually need metadata filtering together with vector similarity — for example:

- search laptops
- under \$1000
- brand = Apple

The challenge is that filters operate on payload fields, not vectors.

When filtering removes many nodes, the HNSW graph can lose connectivity, making some relevant vectors unreachable during traversal.

There are two naive approaches:

1. **Post-filtering**

First run vector search globally, then apply filters.

This is fast but hurts recall because relevant filtered results may never appear in the top-K candidates.

2. **Pre-filtering**

First filter points, then run HNSW only on the filtered subset.

But restrictive filters fragment the graph, reducing traversal efficiency.

Qdrant solves this using **Filterable HNSW**.

It creates filter-aware graph connectivity by building subgraphs for payload values and maintaining additional edges so traversal still works efficiently under filters.

At query time, Qdrant’s query planner dynamically chooses the best strategy based on filter cardinality:

- If many points match → use HNSW with node skipping.
- If very few points match → perform a full scan over filtered candidates.

To make this efficient, payload fields used in filters should be indexed explicitly, ideally before HNSW construction.”

Agar interviewer deeper pooche

“Why post-filtering is bad?”

Answer:

“Because ANN search returns approximate top-K globally.

After filtering, many results may be discarded, and the actual best filtered result may never have been retrieved.”

“Why pre-filtering is bad?”

Answer:

“HNSW relies on graph connectivity.

Restrictive filtering removes nodes and edges, fragmenting traversal paths and reducing ANN efficiency.”

“What is node skipping?”

Answer:

“During HNSW traversal, Qdrant continues traversing the graph but ignores nodes that do not satisfy the payload filter.”

“When does Qdrant use full scan?”

Answer:

“When filter cardinality is extremely low.

If only a tiny subset matches, scanning those candidates is cheaper than graph traversal.”

Sparse Vector

Sparse Vector kya hota hai?

Sparse vector ek **bahut bada vector (list of numbers)** hota hai jismein:

- **zyadatar values 0 hoti hain**
- sirf kuch positions par non-zero values hoti hain

Isliye ise “**sparse**” (yaani khaali / kam data wala) bola jata hai.

Ab har user ka vector batata hai ki usne kis movie ko rating di.

User_1

[0, 0, 0, 0, 0, 0, 1, 5, 0, 0]

Iska matlab:

- User ne sirf:
 - M7 ko rating **1**
 - M8 ko rating **5**
- Baaki movies ko rate nahi kiya → isliye **0**

Example:

[0,0,0,0,4,0,0,2,0,0]

Efficient sparse format:

indices = [5, 8]

values = [4, 2]

Yaani:

- Movie 5 → rating 4
- Movie 8 → rating 2

Sparse vector mein jo **0** aate hain, woh actually “**no information**” ya “**value missing**” ko represent karte hain.

parse Vectors ko compare kaise karte hain?

Jab do sparse vectors compare karte hain, toh hum dekhte hain:

“Dono vectors kin same features par agree karte hain?”

Movie example mein:

- kya dono users ne same movie dekhi?
- aur kya dono ne us movie ko high rating di?

Dot Product kya hota hai?

Similarity nikalne ke liye **Dot Product** use hota hai.

Formula:

$$A \cdot B = \sum_i A_i B_i$$

Matlab:

- same positions ki values ko multiply karo
- sabko add kar do

Interview Tip

Interviewer ko impress karne ke liye ye line bolna:

“Sparse vectors are efficient because computation happens only on overlapping non-zero dimensions.”

One-Line Definition

“Sparse vectors are memory-efficient high-dimensional representations where only a few dimensions are non-zero, and search is optimized using inverted indexes.”

Real-world Use Cases

Bolna:

- BM25
 - keyword search
 - recommendation systems
 - hybrid search
 - search engines
-

Qdrant-specific Answer

Qdrant mein:

- sparse vectors named vectors ke through configure hote hain
- similarity always dot product hoti hai
- search inverted index se exact hoti hai

Bag-of-Words (BoW) — Hindi Explanation

Bag-of-Words ek simple technique hai jisme hum text ko numbers me convert karte hain.

Isme **sirf words aur unki counting** dekhi jaati hai, sentence ka order nah

“Bag” kyu bolte hain?

Kyuki words ko bas ek bag me daal diya jata hai.

Sentence order important nahi hota.

Example:

- "cheese pizza"
- "pizza cheese"

Dono ka BoW representation same hoga.

Hybrid Search क्या है?

जब हम:

- Dense search
- Sparse search

दोनों को combine करते हैं, उसे Hybrid Search कहते हैं।

Hybrid Search Pipeline

तरीका 1:

Dense Retrieval → Sparse Reranking

पहले AI meaning से documents खोजो।

फिर keyword relevance से rerank करो।

Flow:

User Query



Dense Search



Top 20 Results



Sparse Reranking



Final Results

Advanced Hybrid Search

कई companies multi-stage pipelines बनाती हैं:

Stage 1

Fast retrieval

Stage 2

Better reranking

Stage 3

Business rules

Stage 4

Cross encoder / ColBERT

असिन Summary	
Search Type	Best For
Dense Search	Meaning समझना
Sparse Search	Exact keyword match
Hybrid Search	दोनों का balance

Short Interview Explanation (Best Version)

“Hybrid search combines semantic search (dense vectors) and keyword search (sparse vectors) to improve retrieval quality.

Dense vectors understand meaning and intent, while sparse vectors handle exact keyword matching.

This helps serve both users who use natural language queries and users who search with precise terms.

In systems like Qdrant, we can combine both approaches using techniques like Reciprocal Rank Fusion (RRF), where results from multiple retrieval methods are merged based on ranking rather than raw scores.”

If Interviewer Asks:

“Why not only dense search?”

You can say:

“Dense search is great for semantic understanding, but it may miss exact identifiers, model numbers, legal sections, or product codes.

Sparse search handles those exact matches better.”

If They Ask:

“Why not only sparse search?”

“Sparse search depends heavily on exact keywords.

If users describe something vaguely or use synonyms, sparse retrieval may fail.

Dense vectors solve that by understanding context and meaning.”

Best One-Line Definition

“Hybrid search balances semantic understanding and exact keyword matching by combining dense and sparse retrieval techniques.”

Architecture Explanation (Very Important)

अगर interviewer system design perspective से पूछे:

User Query



Dense Retrieval



Sparse Retrieval



Fusion / Reranking

↓
Final Ranked Results

फिर बोलो:

“Usually we retrieve candidates from both methods and combine them using fusion techniques like RRF.”

RRF Explain in One Line

“RRF combines rankings from multiple retrievers instead of comparing their raw scores directly.”

RRF (Reciprocal Rank Fusion) is a ranking fusion algorithm used to combine results from multiple retrieval systems, such as BM25 and vector search. It ranks documents based on their positions in multiple ranked lists rather than raw scores, improving hybrid retrieval performance

RRF (Reciprocal Rank Fusion) ka kaam hai **alag-alag search systems ke results ko mila kar better final ranking banana**.

Simple Hindi me:

- Maan lo ek search system keyword se search karta hai (BM25)
- Aur doosra meaning/similarity se search karta hai (Vector Search)

Dono alag order me documents dete hain.

RRF in dono lists ko combine karta hai aur dekhta hai ki **kaunsa document dono systems me consistently upar aa raha hai**.

Ye raw score compare nahi karta, sirf **rank/position** dekhta hai.

Example:

- BM25 me Doc A rank 1 pe hai
- Vector Search me Doc A rank 3 pe hai

Toh RRF boleگا:

“Doc A dono jagah top ke paas aa raha hai, isliye isko final result me upar rakho.”

Isse hybrid search zyada accurate aur balanced ho jata hai.

Final summary

Type	Bits	Bytes
float32	32 bits	4 bytes
int8	8 bits	1 byte

Aur:

$$32 \div 8 = 4$$

isliye float32 = 4 bytes aata hai.

Binary Quantization kya karta hai?

Normal vectors me numbers hote hain:

[0.82, -0.44, 0.11, -0.90]

Binary quantization har number ko sirf:

0 or 1

me convert karta hai.

Rule:

Value	Binary
> 0	1
≤ 0	0

Simple Interview Explanation

1. Start with the problem

“Vector databases me embeddings usually float32 format me stored hote hain.
Agar embedding dimension 1536 ho, toh har dimension 4 bytes leta hai.”

Then calculation:

$$1536 \times 4 = 6144 \text{ bytes}$$

“So one vector is roughly 6 KB.
Millions of vectors ke liye memory cost bahut huge ho jaata hai.”

2. Then define Binary Quantization

“Binary quantization is an extreme compression technique where each vector dimension is reduced to a single bit.”

Rule explain:

$$\text{value} > 0 \rightarrow 1$$

$$\text{value} \leq 0 \rightarrow 0$$

Example:

$$[0.8, -0.2, 1.5, -0.7]$$

$$\rightarrow [1, 0, 1, 0]$$

3. Show memory reduction

“Instead of storing 32 bits per dimension, we store only 1 bit.”

Calculation:

Original:

$$1536 \times 32 = 49152 \text{ bits}$$

Binary:

$1536 \times 1 = 1536 \text{ bits}$

Compression:

$\frac{49152}{1536} = 32$

“So we achieve around 32x memory compression.”

4. Explain speed advantage

“Binary vectors allow bitwise operations instead of floating-point computations.”

Mention:

- XOR
- AND
- Hamming distance

“Modern CPUs are highly optimized for bit operations, so similarity search becomes much faster.”

5. Mention tradeoff

“However, binary quantization loses magnitude information because only the sign is preserved.”

Example:

$0.1 \rightarrow 1$

$100 \rightarrow 1$

“So some semantic accuracy is sacrificed.”

6. Best use case

“It works best with high-dimensional embeddings having balanced distributions around zero.”

Examples:

- OpenAI embeddings
 - Cohere embeddings
-

7. Final concise conclusion

“Binary quantization is mainly used in large-scale vector search systems where memory efficiency and throughput are more important than perfect retrieval accuracy.”

Short 30-second version

If interviewer asks briefly:

“Binary quantization compresses float32 embedding vectors into 1-bit representations using sign thresholding. Positive values become 1 and non-positive become 0. This reduces memory usage by roughly 32x and enables very fast bitwise similarity computations, though with some loss in retrieval accuracy.”

Pro Tip

Interviewers usually like hearing these keywords:

- float32
- int8 / bit representation
- memory compression
- SIMD
- bitwise operations
- cosine similarity

- Hamming distance
- retrieval accuracy tradeoff

Use 2–3 naturally in explanation.

Best interview explanation

“Product Quantization compresses vectors by splitting them into smaller sub-vectors and replacing each sub-vector with the index of its nearest learned centroid from a codebook. Instead of storing raw float values, the system stores compact centroid IDs, enabling very high compression ratios such as 32x or 64x. The tradeoff is additional approximation error and slower distance computation.”

One-line intuition

Scalar Quantization compresses numbers.

Binary Quantization compresses signs.

Product Quantization compresses patterns/clusters

Quantization search को fast बनाता है, जबकि oversampling + rescoring accuracy वापस improve करते हैं।

Large-scale vector ingestion requires avoiding naive per-vector upserts because they create huge transaction and indexing overheads. Qdrant solves this using batched and streaming ingestion methods like `upload_collection`, which process data chunk-by-chunk without loading everything into memory.

For scalability, it uses a hybrid architecture where compressed quantized vectors and HNSW indexes stay in RAM for fast search, while original vectors remain on disk. During search, approximate retrieval happens first, followed by rescoring using original vectors for accuracy.”

Traditional Embedding Approach

Transformer encoder हर token (word/subword) के लिए अलग vector बनाता है।

मान लो sentence है:

"Virat Kohli scored a century in Mumbai."

तो model internally कुछ ऐसा बनाता है:

Token	Vector
-------	--------

Virat	v1
-------	----

Kohli	v2
-------	----

scored	v3
--------	----

century	v4
---------	----

Mumbai	v5
--------	----

लेकिन आखिर में इन सबको combine (pool) करके एक **final vector** बना दिया जाता है।

यही single-vector embedding कहलाता है।

Late Interaction Models (जैसे ColBERT)

Traditional embeddings

“पूरे document का average meaning”

ColBERT late interaction

“Query के हर word के लिए document में सबसे relevant part ढूँढो”

Simple Interview Answer

Traditional embedding models convert an entire document into a single vector.

This works well for simple semantic search, but in long or multi-topic documents, important token-level details can get lost because everything is compressed into one representation.

ColBERT solves this using **late interaction**.

Instead of storing one vector per document, ColBERT stores embeddings for each token separately. So a document becomes a collection of token vectors (multivectors).

Simple Interview Answer

Traditional embedding models convert an entire document into a single vector.

This works well for simple semantic search, but in long or multi-topic documents, important token-level details can get lost because everything is compressed into one representation.

ColBERT solves this using **late interaction**.

Instead of storing one vector per document, ColBERT stores embeddings for each token separately. So a document becomes a collection of token vectors (multivectors).

Tradeoff

Because every token embedding is stored and compared:

- Storage cost increases
- Search becomes computationally expensive
- Brute-force similarity is slower than single-vector ANN search

So in production, people often use:

Qdrant Connection

Qdrant supports this using **multivectors** and **MAX_SIM comparator**.

A document is stored as multiple token vectors, and Qdrant performs late-interaction scoring during query time.

```
page.get_text()
```

जिसका काम है:

➡ उस PDF page का text निकालना।

“BM25 is a keyword-based ranking algorithm that scores and ranks documents using term frequency, inverse document frequency, and document length normalization.”

Interview me RRF ko aise explain kar sakte ho:

“RRF (Reciprocal Rank Fusion) is a ranking fusion algorithm used to combine results from multiple retrievers like BM25 and vector search based on their ranks instead of raw score

Why use RRF?”

Answer:

“Because BM25 and vector search produce different score scales, directly comparing scores is unreliable. RRF solves this by combining ranks instead of scores.”

Python me **for loop** bahut important hai.
Almost har program me use hota hai.

Simple language me:

“Repeat karne ke liye use hota hai.”

for loop

Use karo jab:

kitni baar loop chalega
already pata ho

while loop

Use karo jab:

loop kab stop hoga
condition decide kare

Important Question

Tum pooch rahe ho probably:

“Nested loop kab use karte hain?”

Ye samjho 🙌

Nested Loop Kab Use Hota Hai?

Jab:

har item ke andar
aur items ho

ya

rows aur columns ho

Real Life Example

School

3 classes
har class me 5 students

Yaha nested loop perfect hai.

Visualization

Class 1

Student 1

Student 2

Student 3

Class 2

Student 1

Student 2

Student 3

Programming Example

```
for class_no in range(3):  
    for student in range(5):  
        print(class_no, student)
```

Patterns Me

Nested loops MOSTLY patterns me use hote hain.

Example

```
for i in range(3):  
    for j in range(4):  
        print("", end="")
```

```
    print()
```

Output:

```
****
```

```
****
```

Python mein `!=` ka matlab hota hai "**not equal to**" yani "**barabar nahi hai**".

Ye comparison operator hai jo check karta hai ki do values alag hain ya nahi.

Example 1

```
a = 5
```

```
b = 3
```

```
print(a != b)
```

Output:

True

Kyuki 5 aur 3 equal nahi hain.

Example 2

```
x = 10
```

```
y = 10
```

```
print(x != y)
```

Output:

False

Kyuki dono values same hain.

Retrieval-augmented generation (RAG) 612

Retrieval-augmented generation (RAG) integrates external information retrieval into the process of generating responses by Large Language Models (LLMs). It searches a database for information beyond its pre-trained knowledge base, significantly improving the accuracy and relevance of the generated responses

Hybrid search

However, neither keyword search nor vector search are always perfect. Keyword search may miss relevant information expressed differently, while vector search can sometimes struggle with specificity or neglect important statistical word patterns. Hybrid methods aim to combine the strengths of different techniques.

Retrieval → pehle information dhundna

Augmented → us information ko add/enhance karna

Generation → final answer generate karna

Retrieval (Khojna)

User ka question aata hai, aur system relevant documents/data search karta hai.

Augmentation (Jodna)

Jo information mili, use prompt ke saath LLM ko diya jata hai.

Generation (Jawab banana)

LLM retrieved information ke basis par answer generate karta hai.

RAG stands for Retrieval-Augmented Generation.

It is an AI architecture where the system first retrieves relevant information from external data sources like PDFs, databases, or vector stores, and then sends that context to an LLM to generate an accurate response.

This helps reduce hallucinations and allows the model to answer using up-to-date or private company data.”

Good interview point:

“RAG is cost-effective compared to fine-tuning because we can update knowledge without retraining the model.”

Indexing - Indexing is the process of preparing your knowledge base so that it can be efficiently searched at query time. This steps consists of 4 sub-steps.

1. Document Ingestion - You load your source knowledge into memory.

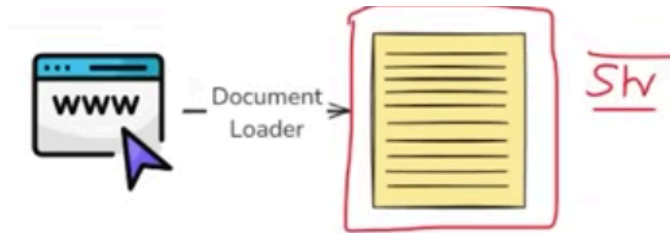
🔥 Examples:

- PDF reports, Word documents
- YouTube transcripts, blog pages
- GitHub repos, internal wikis
- SQL records, scraped webpages

🔧 Tools:

- LangChain loaders (PyPDFLoader, YoutubeLoader, WebBaseLoader, GitLoader, etc.)





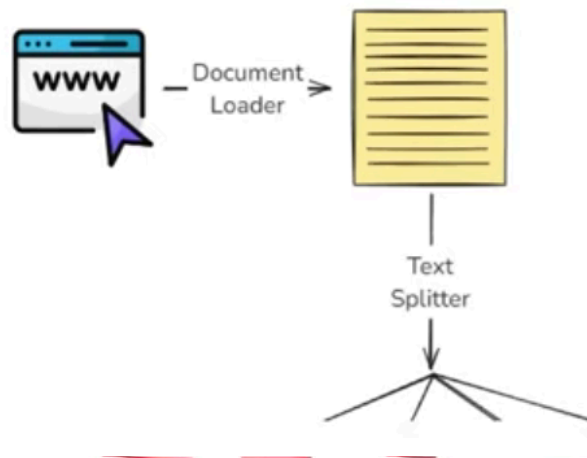
2. Text Chunking - Break large documents into small, semantically meaningful chunks

Why chunk?

- LLMs have context limits (e.g., 4K-32K tokens)
- Smaller chunks are more focused → better semantic search

Tools:

- `RecursiveCharacterTextSplitter`, `MarkdownHeaderTextSplitter`, `SemanticChunker`



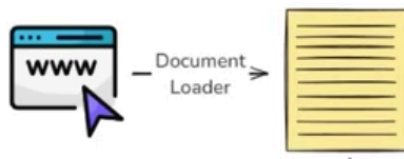
3. Embedding Generation - Convert each chunk into a dense vector (embedding) that captures its meaning.

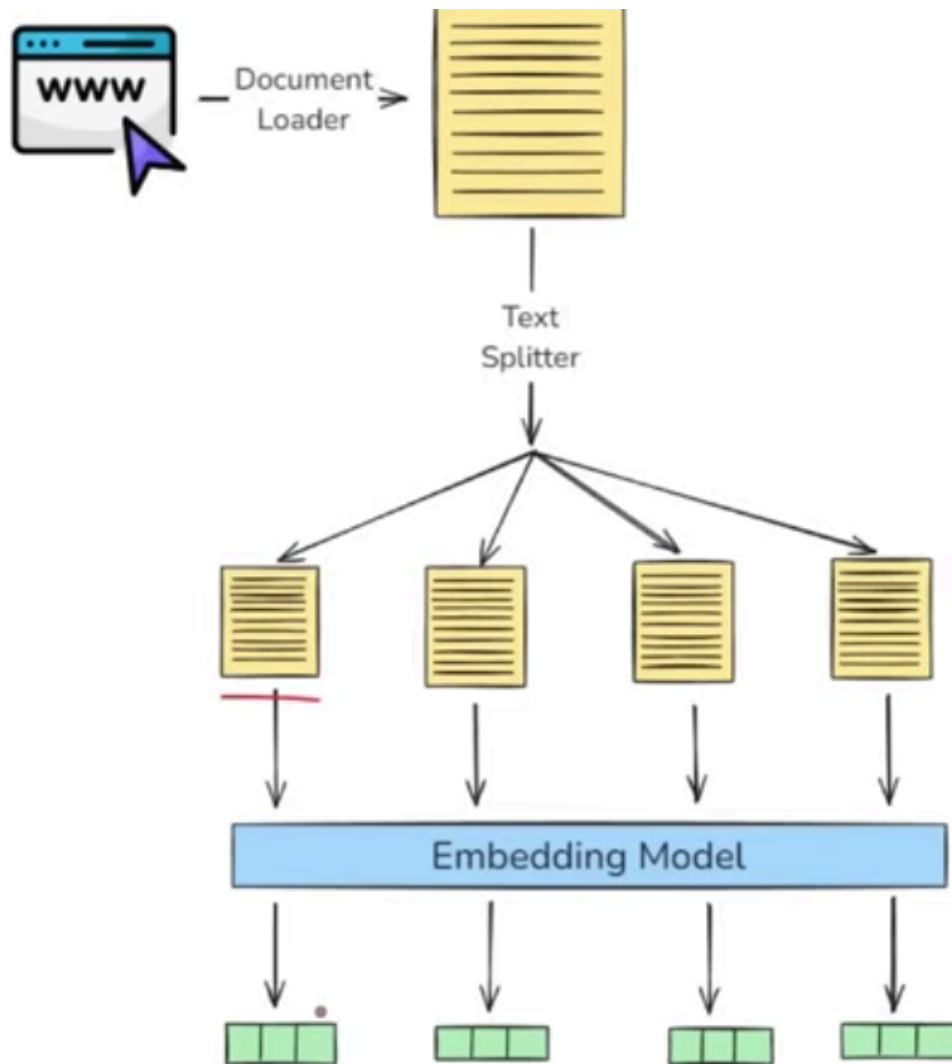
Why embeddings?

- Similar ideas land close together in vector space
- Allows fast, fuzzy semantic search

Tools:

- `OpenAIEmbeddings`, `SentenceTransformerEmbeddings`, `InstructorEmbeddings`, etc.

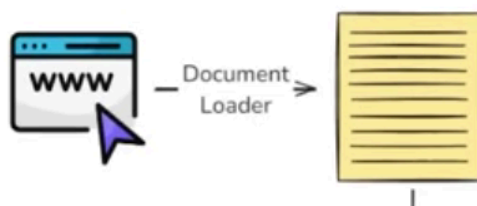


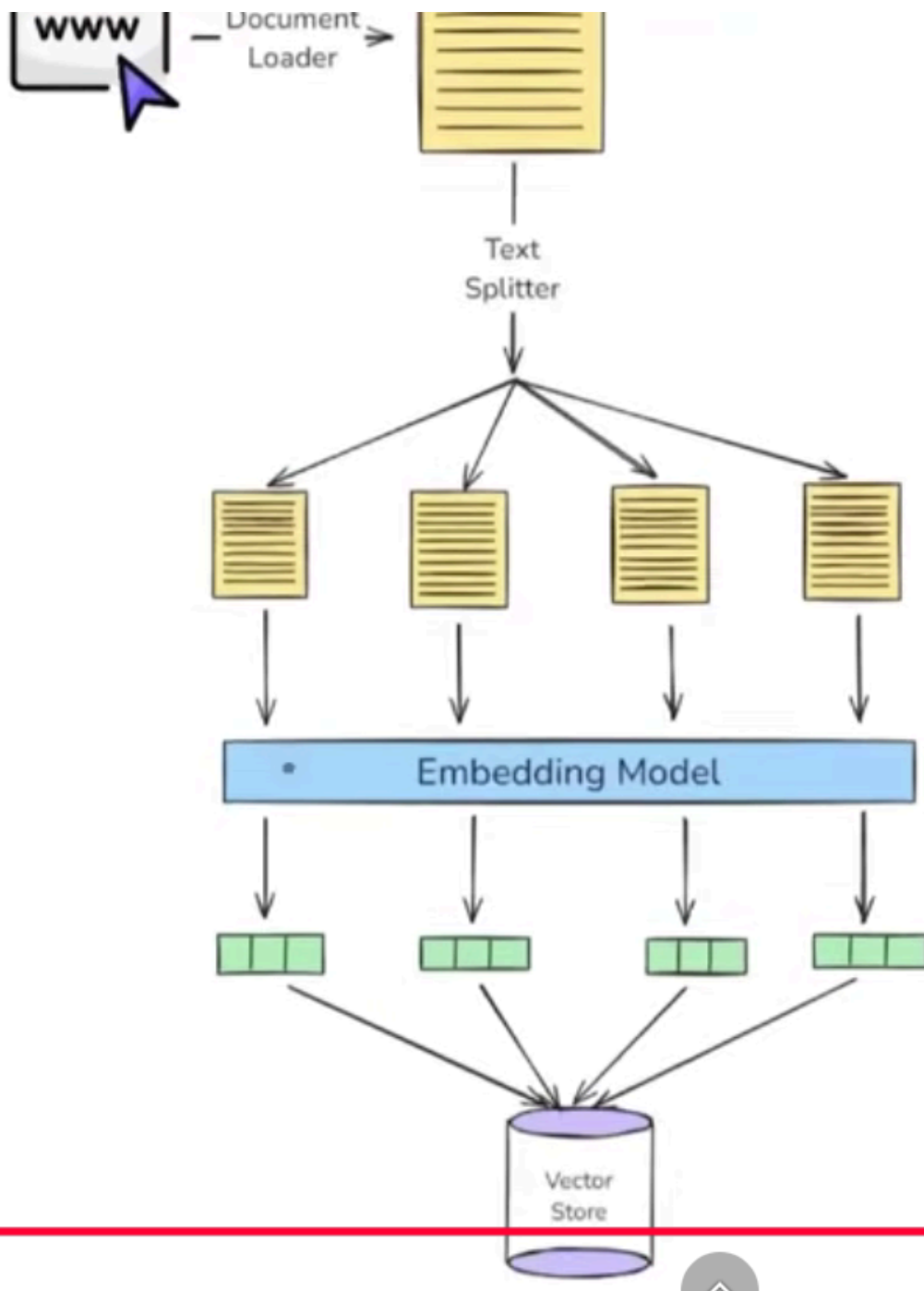


4. Storage in a Vector Store - Store the vectors along with the original chunk text + metadata in a vector database.

🔑 Vector DB options:

- Local: FAISS, Chroma
- Cloud: Pinecone, Weaviate, Milvus, Qdrant

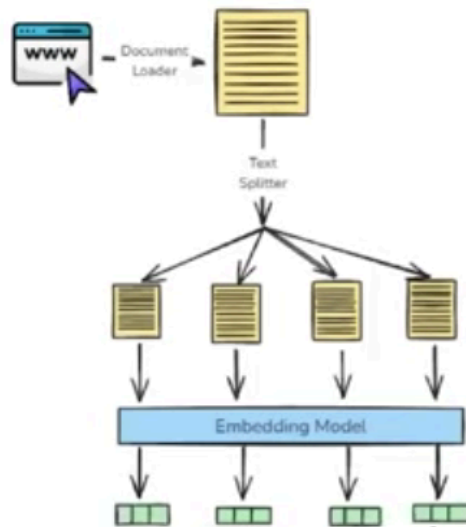


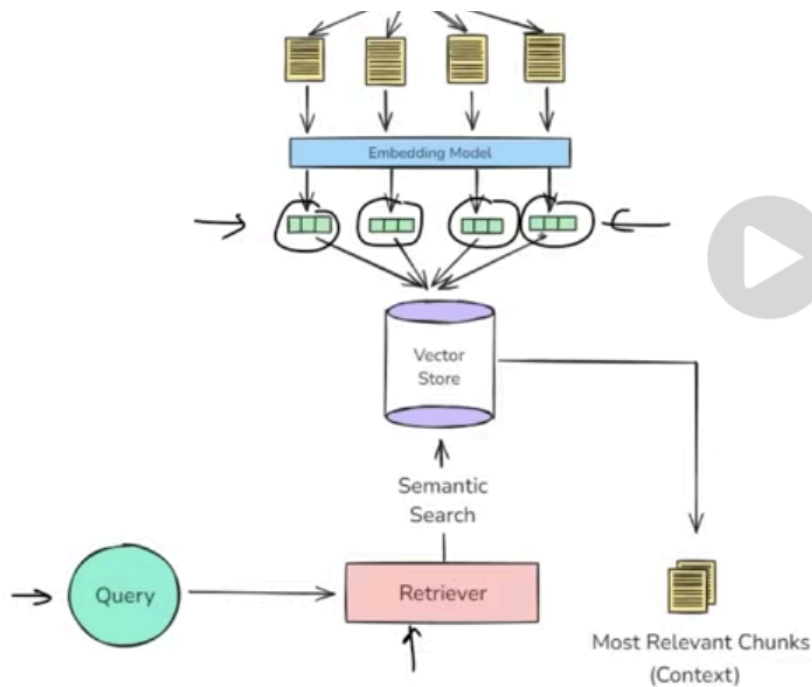


Retrieval - **Retrieval** is the *real-time* process of finding the most relevant pieces of information from a pre-built index (created during indexing) based on the user's question.

It's like asking:

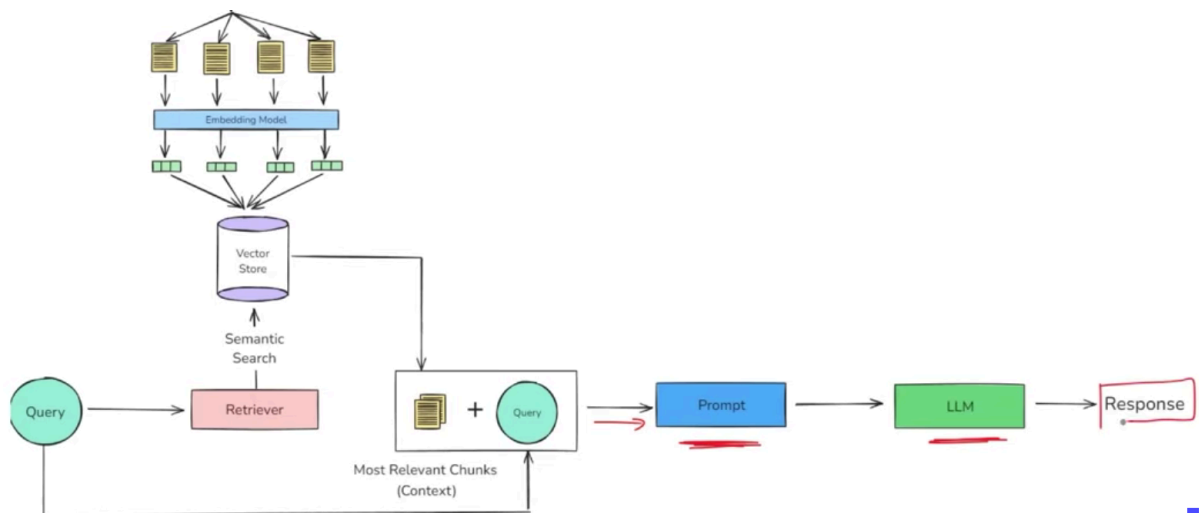
"From all the knowledge I have, which 3-5 chunks are most helpful to answer this query?"





Step 1 → Que
Step 2 →

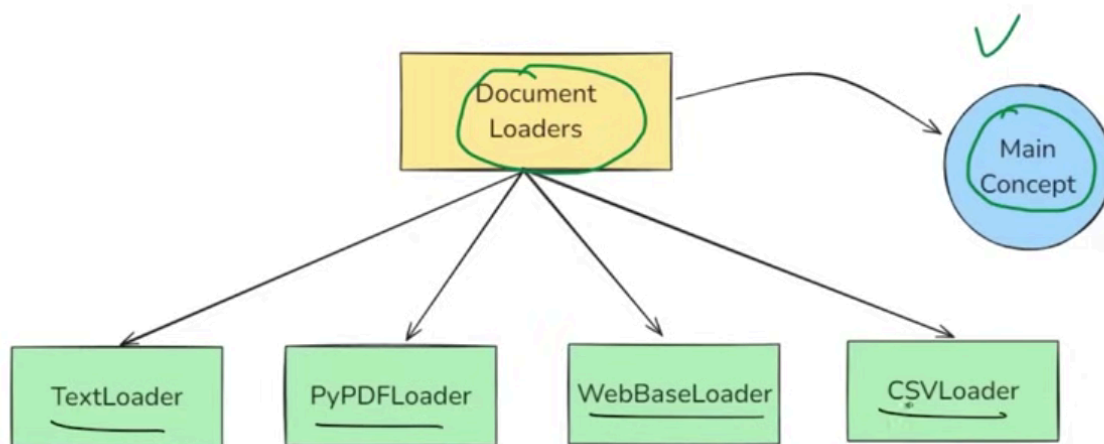
Augmentation - **Augmentation** refers to the step where the **retrieved documents** (chunks of relevant context) are **combined with the user's query** to form a new, enriched prompt for the LLM.



RAG is a technique that combines information retrieval with language generation, where a model retrieves relevant documents from a knowledge base and then uses them as context to generate accurate and grounded responses.

Benefits of using RAG

1. Use of up-to-date information
2. Better privacy
3. No limit of document size

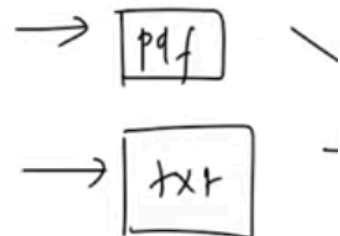


Document Loaders in LangChain

27 March 2025 16:20

Document loaders are components in LangChain used to load data from various sources into a standardized format (usually as Document objects), which can then be used for chunking, embedding, retrieval, and generation.

```
Document(  
    page_content="The actual text content",  
    metadata={"source": "filename.pdf", ...}  
)
```



PyPDFLoader

27 March 2025 17:50

PyPDFLoader is a document loader in LangChain used to load content from PDF files and convert each page into a Document object.

```
[
    Document(page_content="Text from page 1", metadata={"page": 0, "source": "file.pdf"}),
    Document(page_content="Text from page 2", metadata={"page": 1, "source": "file.pdf"}),
    ...
]
```

Limitations:

- It uses the PyPDF library under the hood — not great with scanned PDFs or complex layouts.

Use Case	Recommended Loader
Simple, clean PDFs	PyPDFLoader
PDFs with tables/columns	PDFPlumberLoader
Scanned/image PDFs	UnstructuredPDFLoader or AmazonTextractPDFLoader
Need layout and image data	PyMuPDFLoader
Want best structure extraction	UnstructuredPDFLoader

DirectoryLoader

27 March 2025 18:44

DirectoryLoader is a document loader that lets you load multiple documents from a directory (folder) of files.

Glob Pattern	What It Loads
"**/*.txt"	All .txt files in all subfolders
"*.pdf"	All .pdf files in the root directory
"data/*.csv"	All .csv files in the data/ folder
"**/*"	All files (any type, all folders)

****** = recursive search through subfolders

Load vs Lazy load

27 March 2025 23:51

load()

- **Eager Loading** (loads everything at once).
- Returns: A list of `Document` objects.
- Loads all documents **immediately** into memory.
- Best when:
 - The number of documents is small.
 - You want everything loaded upfront.

lazy_load()

- **Lazy Loading** (loads on demand).
- Returns: A **generator** of `Document` objects.
- Documents are **not all loaded at once**; they're fetched one at a time as needed.
- Best when:
 - You're dealing with **large documents or lots of files**.
 - You want to **stream** processing (e.g., chunking, embedding) without using lots of memory.

WebBaseLoader

28 March 2025

00:34

WebBaseLoader is a document loader in LangChain used to load and extract text content from web pages (URLs).

It uses BeautifulSoup under the hood to parse HTML and extract visible text.

When to Use:

- For blogs, news articles, or public websites where the content is primarily text-based and static.

Limitations:

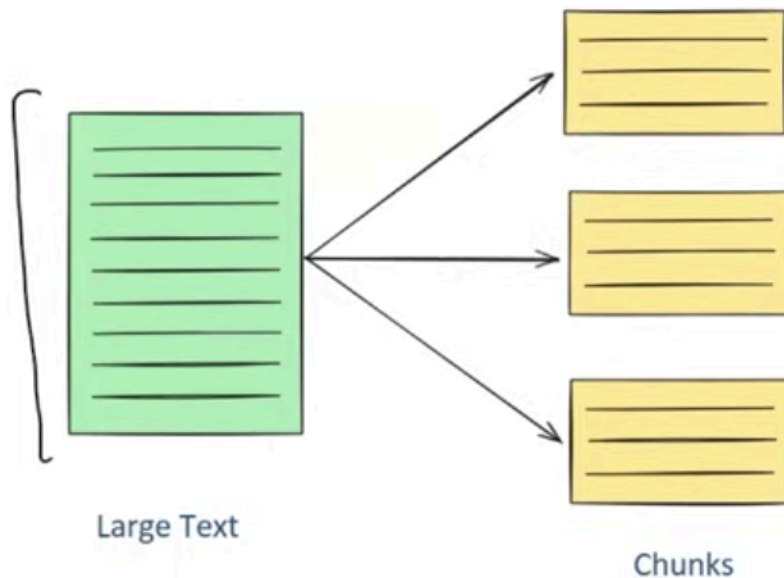
- Doesn't handle JavaScript-heavy pages well (use SeleniumURLLoader for that).
- Loads only static content (what's in the HTML, not what loads after the page renders).

Text Splitting

01 April 2025

18:10

Text Splitting is the process of breaking large chunks of text (like articles, PDFs, HTML pages, or books) into smaller, manageable pieces (chunks) that an LLM can handle effectively.



- Overcoming model limitations: Many embedding models and language models have maximum input size constraints. Splitting allows us to process documents that would otherwise exceed these limits.

- Downstream tasks - Text Splitting improves nearly every LLM powered task

Task	Why Splitting Helps
Embedding	Short chunks yield more accurate vectors
Semantic Search	Search results point to focused info, not noise
Summarization	Prevents hallucination and topic drift

- Optimizing computational resources: Working with smaller chunks of text can be more memory-efficient and allow for better parallelization of processing tasks.

What are Retrievers

10 April 2025 07:56

A retriever is a component in LangChain that fetches relevant documents from a data source in response to a user's query.

- There are multiple types of retrievers
- All retrievers in LangChain are runnables

Vector Store Retriever

10 April 2025 16:24

A Vector Store Retriever in LangChain is the most common type of retriever that lets you search and fetch documents from a vector store based on semantic similarity using vector embeddings.

⚙️ How It Works

1. You store your documents in a **vector store** (like FAISS, Chroma, Weaviate)
2. Each document is converted into a **dense vector** using an **embedding model**
3. When the user enters a query:
 - It's also turned into a vector
 - The retriever compares the query vector with the stored vectors
 - It retrieves the top-k most similar ones

If an interviewer asks **"What is RAG?"**, a concise and strong answer is:

RAG (Retrieval-Augmented Generation) is an AI architecture that combines a large language model (LLM) with an external knowledge retrieval system. Instead of relying only on the model's training data, RAG first retrieves relevant information from documents, databases, or knowledge bases and then uses that information to generate a more accurate and up-to-date response.

R = Retrieval

Retrieval means finding or fetching relevant information from a knowledge source (documents, PDFs, database, vector database, etc.).

A = Augmented

Augmented means enhancing or enriching the prompt with the retrieved information.

Question + Retrieved Context

G = Generation

Generation means the LLM generates the final answer using the augmented context.

Example:

- Based on the retrieved policy, the LLM generates:

"Employees are entitled to 20 paid leaves per year."

Simple Flow

1. User asks a question.
2. The question is converted into an embedding.
3. A vector database searches for relevant documents.
4. The retrieved documents are added to the prompt as context.
5. The LLM generates an answer based on both the query and the retrieved context.

Why Use RAG?

- Reduces hallucinations.
- Provides answers based on company-specific or latest data.
- No need to retrain the model whenever documents change.
- Improves accuracy and traceability.

Interview Example

Suppose you build a chatbot for a company's HR policies.

Without RAG:

- The LLM may not know the latest leave policy.

- It may generate incorrect information.

With RAG:

- The system retrieves the latest HR policy document.
- The LLM answers using the retrieved policy.
- Responses stay accurate even when policies are updated

What is LangChain

29 January 2025 08:35

LangChain is an open source framework that helps in building LLM based applications. It provides modular components and end-to-end tools that help developers build complex AI applications, such as chatbots, question-answering systems, retrieval-augmented generation (RAG), autonomous agents, and more.

- ConversationBufferMemory: Stores a transcript of recent messages. Great for short chats but can grow large quickly.
- ConversationBufferWindowMemory: Only keeps the last N interactions to avoid excessive token usage.
- Summarizer-Based Memory: Periodically summarizes older chat segments to keep a condensed memory footprint.
- Custom Memory: For advanced use cases, you can store specialized state (e.g., the user's preferences or key facts about them) in a custom memory class.

What are Models

07 January 2025 23:15

The Model Component in LangChain is a crucial part of the framework, designed to facilitate interactions with various language models and embedding models.

It abstracts the complexity of working directly with different LLMs, chat models, and embedding models, providing a uniform interface to communicate with them. This makes it easier to build applications that rely on AI-generated text, text embeddings for similarity search, and retrieval-augmented generation (RAG).

Detailed Interview Answer (2–3 Minutes)

"An LLM Agent in LangChain is a framework that enables a Large Language Model to act as a decision-maker rather than just a text generator.

The LLM receives a user query, reasons about the task, and determines whether it needs additional information or actions. If required, it selects an appropriate tool, such as a web search tool, calculator, database query, or API call. After receiving the tool's output, it evaluates the result and may perform additional steps before producing the final answer.

The key components are:

1. **LLM** – provides reasoning and decision-making.
2. **Tools** – external functions or APIs the agent can use.
3. **Prompt** – instructions that define the agent's behavior.
4. **Memory** – stores conversation context when needed.
5. **Agent Executor** – manages the reasoning and execution cycle.

What is Agent

Simple Definition

An agent is an entity that takes input from its environment, makes decisions, and performs actions to achieve a specific objective.

What is an agent?", keep it practical and concise:

"An agent is an AI system that can think, make decisions, and use tools to complete a task. Instead of just generating text, it can choose actions, call APIs, search for information, query databases, or perform calculations, and then use the results to provide the final answer."

Why do we use agents?

"We use agents when a task requires multiple steps or external information. A normal LLM only generates answers from its knowledge, but an agent can interact with tools and systems to get real-time data and complete complex tasks automatically."

```
def get_weather(city : str):
    """Get Weather for a give city"""
    return f"it always sunny in{city}"

agent = create_agent(
    model=llm,
    tools=[get_weather],
    system_prompt="You are a helpful assistant"
)

result = agent.invoke(
    {"messages":[{"role" : "user", "content":"What's the weather in San Francisco?"}]}
)

print(result["messages"][-1].content_blocks)
```

```
content_blocks का काम है message को structured format में देना, सिर्फ plain text नहीं।
```

Define the system prompt

The system prompt defines your agent's role and behavior. Keep it specific and actionable:

```
SYSTEM_PROMPT = """You are a literary data assistant.
```

```
## Capabilities
```

```
- `fetch_text_from_url`: loads document text from a URL into the conversation.
```

```
Do not guess line counts or positions—ground them in tool results from the saved file."""
```

Create tools

Tools let a model interact with external systems by calling functions you define. Tools can depend on runtime context and also interact with agent memory.

This example uses a tool to load a document from a given URL:

```
import urllib.error
import urllib.request
```

```
from langchain.tools import tool
```

```
@tool
```

```
def fetch_text_from_url(url: str) -> str:
    """Fetch the document from a URL.
    """
    req = urllib.request.Request(
        url,
        headers={"User-Agent": "Mozilla/5.0 (compatible;
quickstart-research/1.0)"},
    )
    try:
        with urllib.request.urlopen(req, timeout=120) as resp:
            raw = resp.read()
    except urllib.error.URLError as e:
        return f"Fetch failed: {e}"
    text = raw.decode("utf-8", errors="replace")
    return text
```

```
model = init_chat_model(
    "gemini-3.1-pro-preview",
    model_provider="google-genai",
    temperature=0.5,
    timeout=600,
    max_tokens=25000,
    streaming=True,
)
```

3. Temperature

temperature=0.5

Response kitna creative hoga.

- 0.0 → bilkul deterministic, same type ke answers
- 0.5 → balanced
- 1.0+ → zyada creative/random

Timeout

`timeout=600`

600 seconds (10 minutes) tak model response ka wait karega.

Agar isse zyada time laga to request fail ho sakti hai.

Max Tokens

`max_tokens=25000`

Model maximum 25,000 tokens generate kar sakta hai.

Approximation:

- 1 token $\approx \frac{3}{4}$ word
- 25,000 tokens $\approx$ 18,000–20,000 words

Long reports aur research outputs ke liye useful.

6. Streaming

`streaming=True`

Response poora generate hone ka wait nahi karega.

Jaise ChatGPT typing karta hua dikhta hai:

Hello...

How can I help...

waise chunks mein response aayega.

Add memory

Add memory to your agent to maintain state across interactions. This allows the agent to remember previous conversations and context.

LangChain Agent kya hota hai?

Agent = LLM + Tools + Control Loop

LangChain Agent kya hota hai?

Agent = **LLM + Tools + Control Loop**

"An Agent in LangChain is an AI system that combines a language model with tools. Instead of answering directly, it can decide when to use a tool, get the required information, and then generate the final answer. It keeps repeating this process until the task is completed."

What is a Harness?

"A harness is the framework around the agent. It includes the model, prompts, tools, memory, and execution logic that help the agent perform tasks."

Thread ID is used for conversation persistence and memory management, while context is used to pass runtime information such as user ID, API keys, or feature flags to tools and middleware. Thread ID maintains chat history, whereas context carries request-specific metadata

Suppose you are building a banking chatbot:

```
thread_id = "chat_123"  
context = Context(  
    user_id="user_789",  
    account_type="premium"  
)
```

Explain:

"The `thread_id` helps the bot remember previous messages in the same banking conversation. The `context` provides additional runtime information such as which user is making the request and whether they have a premium account. Tools can use this context to fetch the correct account details."

temperature

number

Controls the randomness of the model's output. A higher number makes responses more creative; lower ones make them more deterministic

max_tokens

number

Limits the total number of tokens in the response, effectively controlling how long the output can be.

timeout

number

The maximum time (in seconds) to wait for a response from the model before canceling the request.

.

SystemMessage → Rules/instructions

HumanMessage → User ka message

AIMessage → Assistant ka previous response

What is Models

In LangChain, a model is the Large Language Model responsible for generating responses. LangChain acts as an orchestration framework, while the model performs the actual reasoning and text generation.

LangChain provides a unified interface so we can switch between providers like OpenAI, Google Gemini, and Anthropic Claude with minimal code changes.

Configurable models -

A configurable model is a model whose provider and model name can be selected at runtime through configuration instead of being fixed when the

agent is created. This enables dynamic model routing, cost optimization, and flexibility across multiple LLM providers.

```
from langchain.chat_models import init_chat_model
```

```
configurable_model = init_chat_model(temperature=0)
```

```
configurable_model.invoke(
    "what's your name",
    config={"configurable": {"model": "gpt-5-nano"}}, # Run with GPT-5-Nano
)
configurable_model.invoke(
    "what's your name",
    config={"configurable": {"model": "claude-sonnet-4-6"}}, # Run with
Claude
)
```

Configurable models

You can also create a runtime-configurable model by specifying `configurable_fields`. If you don't specify a model value, then `model` and `model_provider` will be configurable by default.

Dynamic model selection

Dynamic models are selected at runtime based on the current state and context. This enables sophisticated routing logic and cost optimization.

To use a dynamic model, create middleware using the `wrap_model_call` decorator that modifies the model in the request:

Dynamic Model Selection ka matlab hai ki AI application automatically decide karegi ki kis model ko use karna hai, based on current situation, user request, ya context.

Hinglish Explanation:

Runtime par (jab request aa rahi ho) system current situation aur context ko dekhkar automatically best model choose karta hai.

Configurable Models

"Configurable models allow us to change the underlying LLM at runtime without changing the application code. Instead of hardcoding a model like GPT-5, we make the model configurable and pass the desired model through configuration."

Dynamic Model Selection

"Dynamic model selection is an advanced routing mechanism where the application automatically chooses the most appropriate model based on the request context"

Messages

system → AI ko instructions dene ke liye

user → User ka question ya input

assistant → AI ka response

SystemMessage is used to prime the model's behavior by defining its role, tone, rules, and response guidelines before the conversation starts.

AIMessage manually kyu create karte hain?

Documentation ka ye part important hai:

"Sometimes it is helpful to manually create a new AIMessage object and insert it into the message history."

Matlab kabhi-kabhi hum khud AI ka purana response history mein add kar sakte hain.

Tools

A tool is a function that an LLM can call to perform tasks outside of its built-in knowledge, such as fetching real-time data, querying databases, executing code, or interacting with external APIs.

Stream writer

Stream real-time updates from tools during execution. This is useful for providing progress feedback to users during long-running operations

Execution info provides runtime metadata such as thread ID, run ID, and retry state within a tool. It helps with tracking, debugging, logging, monitoring executions, and handling retries in agent workflows.

Short-term memory -

Documentation:

Short term memory lets your application remember previous interactions within a single thread or conversation.

Matlab:

Short-term memory sirf current conversation/thread ke andar kaam karti hai.

A checkpointer enables thread-level memory, allowing an agent to remember conversation history within a chat session.

Customizing agent memory

Customizing agent memory means extending AgentState to store application-specific data in addition to conversation history.

```
class CustomAgentState (AgentState) :  
    user_id: str
```

```
preferences: dict
```

When short-term memory grows too large, it can exceed the LLM's context window and degrade performance. Common solutions include trimming old messages, summarizing conversation history, forgetting stale information, and moving important information to long-term memory with retrieval when needed.

`runtime.state` allows tools to access the agent's short-term memory while keeping memory management hidden from the model

Event Streaming

Agar **event streaming** ho, to answer bante hi thoda-thoda data aata rahega.

Event streaming is the real-time flow of events from producers to consumers, allowing applications to process and react to data as soon as it is generated rather than waiting for batch or final results.

"`message.tool_calls` streams the tool call arguments as they are generated by the LLM before the actual tool execution starts."

"LangChain exposes two tool-call streams. `message.tool_calls` streams tool-call arguments while the LLM is constructing the tool request. `stream.tool_calls` streams the execution lifecycle after the tool starts, including start, progress, completion, and results. In short, the first shows what the model wants to execute, and the second shows what actually happens during execution

"Streaming sub-agents" ka matlab

Jab sub-agent kaam kar raha hota hai aur streaming enabled hai, to uske events bhi stream mein aate rehte hain.

Matlab sirf main agent ke messages nahi milenge, balki sub-agent kya kar raha hai woh bhi dekh sakte ho.

`asyncio.gather()` is used to run multiple asynchronous tasks concurrently and wait for all of them to complete. Instead of executing tasks one after another, it schedules them together, improving efficiency for I/O-bound operations.

Bahut accha question.

`asyncio.gather()` bhi wait karta hai, lekin difference ye hai ki **kis cheez ka wait karta hai**.

Without `gather`

`await task1()`

`await task2()`

Yahan:

1. `task1` start
2. `task1` complete hone ka wait
3. `task2` start
4. `task2` complete hone ka wait

Middleware can register stream transformer factories through the `transformers` attribute. The framework calls each factory with a scope tuple representing the current graph or subgraph and expects a new `StreamTransformer` instance in return. Creating a fresh transformer per scope ensures state isolation between the root graph and nested subgraphs.

A Stream Transformer is a component that sits in the event stream pipeline and processes events before they are delivered downstream. It can modify, filter, enrich, log, or forward streaming events, enabling custom behavior without changing the agent's core logic.

Interview Answer

`write()` is used by a stream transformer to emit an event to the next stage of the stream pipeline. After inspecting, modifying, or filtering an event, the transformer calls **`write()`** to forward the event downstream. If **`write()`** is not called, that event typically does not continue through the stream.

Simple Comparison	
Streaming	Event Streaming
Output/text stream hota hai	Events stream hote hain
Usually tokens/chunks	Messages, tool calls, state updates, errors, etc.
User response dekhne ke liye	Monitoring, debugging, orchestration ke liye
Simpler	More detailed

Interview answer:

Streaming delivers output incrementally, usually as text chunks or tokens. Event streaming delivers structured events such as tool calls, state updates, message lifecycle events, and outputs, giving visibility into the entire execution process rather than just the final response.

Streaming in LangChain allows responses and execution updates to be delivered incrementally instead of waiting for the entire operation to complete. This improves responsiveness, reduces perceived latency, and provides a better user experience, especially when LLMs, tools, or agents take several seconds to finish processing.

Normal Streaming ka Purpose

Sirf output dikhana.

AI:

Hello...

How are...

you...

User ko response jaldi dikh jata hai.

Event Streaming ka Purpose

Agent ke andar kya ho raha hai woh dikhana.

Example:

User: Find latest AI news

Agent internally:

1. Search tool call
2. Search results received
3. Summarization started
4. Response generated

Event streaming ye sab expose karta hai.

1. Normal Streaming (**stream**, token streaming)

Use jab user ko **sirf response dikhana ho**.

Example:

- Chatbot
- Q&A app
- Customer support bot
- ChatGPT jaisa UI

2. Event Streaming (**astream_events**)

Use jab tumhe **agent ke internal actions bhi dekhne hain**.

Example:

-  Searching documents
-  Calling tool
-  Retrieved 20 records
-  Summarizing
-  Done

Ya debugging karni ho.

-  Use **event streaming**

5. Multiple Streaming Modes	
LangChain mein different modes available hain:	
Mode	Kya Stream Hota Hai
updates	Agent ke steps/progress
messages	LLM ke generated tokens + metadata
custom	Aapke custom events/data

"Custom updates provide live progress information during agent execution. Using `get_stream_writer()`, we can stream status messages to improve user experience."

"Streaming means receiving the LLM output incrementally instead of waiting for the full response. In LangChain, we can stream normal text tokens, reasoning tokens, tool-call chunks, and tool responses. Using `stream_mode='messages'` gives real-time tokens, while

`stream_mode='updates'` gives completed tool executions and state updates. This improves responsiveness and visibility into agent execution."

"LangChain streaming allows real-time delivery of LLM outputs. Using `stream_mode='messages'`, we can stream text tokens, reasoning tokens, and tool-call chunks as they are generated. Using `stream_mode='updates'`, we receive completed tool executions and state updates. We can also stream custom events from middleware using `stream_mode='custom'`. This improves user experience, observability, debugging, and monitoring of agent workflows. In production, we commonly use `stream_mode=['messages', 'updates']` to get both live tokens and completed execution events."

Ticket_ID
Age
Ticket_Price
Movie_Genre
Seat_Type
Number_of_Person
Purchase_Again

1. SELECT = Data Dekho

Saara data:

```
SELECT * FROM tickets;
```

Sirf kuch columns:

```
SELECT Ticket_ID, Age  
FROM tickets;
```

Socho:

"Mujhe kya dekhna hai?" → `SELECT`

2. WHERE = Filter Lagao

Sirf Comedy movies:

```
SELECT *  
FROM tickets  
WHERE Movie_Genre = 'Comedy';
```

Sirf VIP seats:

```
SELECT *  
FROM tickets  
WHERE Seat_Type = 'VIP';
```

Age 50 se zyada:

```
SELECT *  
FROM tickets  
WHERE Age > 50;
```

Socho:

"Mujhe kaunsa data chahiye?" → WHERE

3. COUNT = Ginti Karo

Total records:

```
SELECT COUNT(*)  
FROM tickets;
```

Kitni Comedy movies:

```
SELECT COUNT(*)  
FROM tickets  
WHERE Movie_Genre = 'Comedy';
```

Socho:

"Kitne hain?" → COUNT

4. ORDER BY = Sort Karo

Sabse mehngi tickets pehle:

```
SELECT *  
FROM tickets  
ORDER BY Ticket_Price DESC;
```

Sabse sasti pehle:

```
SELECT *  
FROM tickets  
ORDER BY Ticket_Price ASC;
```

Socho:

"Sequence me lagao" → ORDER BY

5. GROUP BY = Category Wise Summary

Har genre me kitne tickets:

```
SELECT Movie_Genre,  
       COUNT(*)  
FROM tickets
```

```
GROUP BY Movie_Genre;
```

Output:

Comedy 1200

Drama 900

Horror 700

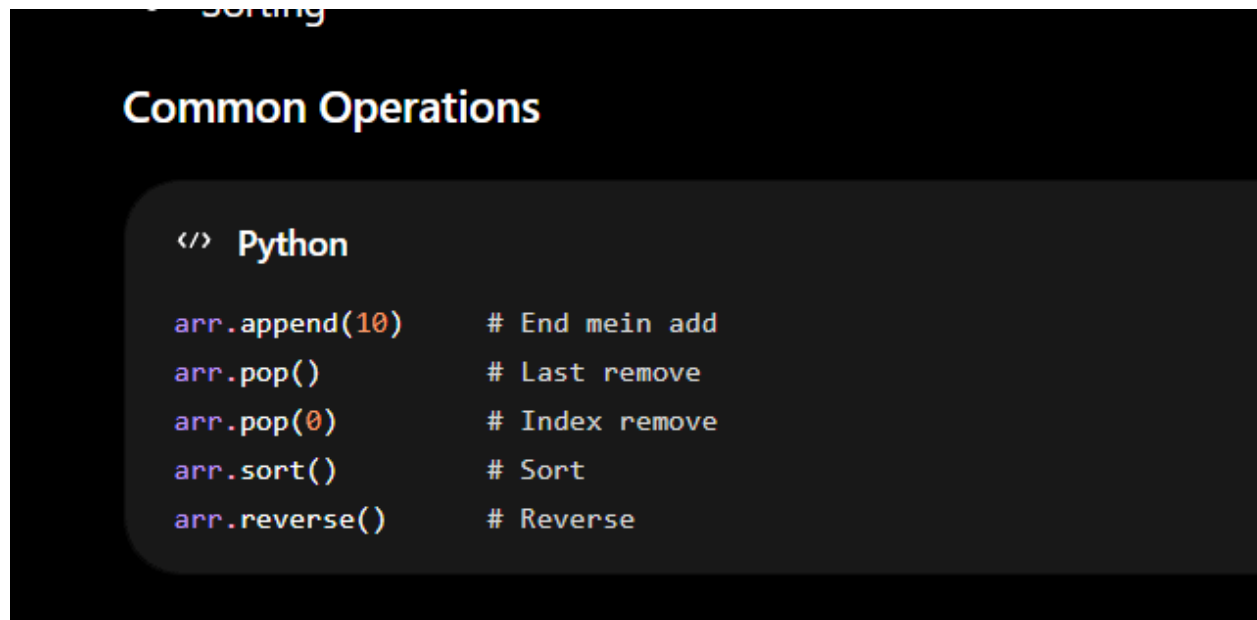
Har seat type ka average price:

```
SELECT Seat_Type,  
       AVG(Ticket_Price)  
FROM tickets  
GROUP BY Seat_Type;
```

Output:

VIP 25.40

Standard 18.20



What is a Multi-Agent System?

A **multi-agent system** is a setup where **multiple AI agents work together**, and each agent specializes in a particular task.

Patterns

Here are the main patterns for building multi-agent systems, each suited to different use cases:

Pattern	How it works
<u>Subagents</u>	A main agent coordinates subagents as tools. All routing passes through the main agent, which decides when and how to invoke each subagent.
<u>Handoffs</u>	Behavior changes dynamically based on state. Tool calls update a state variable that triggers routing or configuration changes, switching agents or adjusting the current agent's tools and prompt.
<u>Skills</u>	Specialized prompts and knowledge loaded on-demand. A single agent stays in control while loading context from skills as needed.
<u>Router</u>	A routing step classifies input and directs it to one or more specialized agents. Results are synthesized into a combined response.
<u>Custom workflow</u>	Build bespoke execution flows with LangGraph , mixing deterministic logic and agentic behavior. Embed other patterns as nodes in your workflow.

What is a Multi-Agent System?

A Multi-Agent System (MAS) is a system where multiple AI agents work together to solve a complex task.

Each agent has a specific responsibility and specialization.

Example:

- Research Agent → Collects information
- Coding Agent → Writes code
- Testing Agent → Tests the code
- Manager Agent → Coordinates everything

2. Why Use Multi-Agent Systems?

Context Management

Different agents handle different knowledge areas, reducing context overload.

Parallel Execution

Multiple agents can work simultaneously, improving speed.

Specialization

Each agent focuses on a specific task and performs better.

Scalability

New agents can be added without changing the entire system.

Most Important Concept: Context Engineering

Context Engineering means deciding:

- What information an agent should see
- What tools an agent can access
- What information should be hidden

Key Point:

The success of a multi-agent system depends on providing the right context to the right agent.

5. Multi-Agent Patterns

A. Subagents Pattern

One main agent manages multiple subagents.

User



Main Agent

├— Research Agent

├— Coding Agent

└— Testing Agent

Use Case: AI assistants, software development agents.

B. Handoff Pattern

Control is transferred from one agent to another.

Support Agent



Technical Issue



Technical Agent

Use Case: Customer support systems.

Q: When should you use a Multi-Agent System?

Answer:

Use a multi-agent system when:

- The task is complex.
- Different expertise is required.
- Parallel execution is beneficial.
- Context becomes too large for a single agent.
- Workflow control and specialization are important.

Q: What is the most important factor in Multi-Agent design?

Answer:

Context Engineering—ensuring each agent receives the right information, tools, and permissions for its task.

 **Interview Tip:** Remember this formula:

Multi-Agent = Specialization + Coordination + Context Engineering + Parallel Execution.

Ek Line Yaad Rakhna

Tool = "Do this task."

Subagent = "Figure out how to do this task and then do it."

Interview Answer

"Tools are simple functions that perform specific actions, whereas subagents are specialized AI agents that can reason, plan, use multiple tools, and solve complex tasks independently. We use subagents when a task requires expertise, multi-step reasoning, or extensive context management."

HANDOFF :-

Handoff is a pattern where the active agent changes dynamically by updating a state variable. Control of the conversation is transferred from one specialized agent to another based on the user's needs or workflow state.

Why use Skills instead of Subagents?

Answer:

Skills are lighter weight. They keep a single agent in control while loading specialized prompts and knowledge on demand. Subagents introduce separate agents and orchestration overhead.

A Skill is an on-demand prompt-based specialization that enhances a single agent's capabilities without creating a separate agent. It enables progressive disclosure by loading only the knowledge needed for the current task.

Interview One-Liner

Router is a lightweight classification layer that directs requests to the most appropriate specialized agent and optionally combines results from multiple agents into a final response.

Why use Router?

Answer:

Router pattern is used when queries belong to distinct domains and need to be directed to specialized agents. It provides lightweight classification and can execute multiple agents in parallel before synthesizing results.

Interview Question

Q. PyMuPDF kyun use kiya?

Answer:

PyMuPDF high-performance PDF parsing library hai. Isse page-wise text extraction, images, metadata aur page

coordinates efficiently extract kiye ja sakte hain. Ye production RAG applications me kaafi popular hai.

What is LangChain vs LangGraph?

LangChain is a higher-level framework with many ready-made components. It makes it easy to build common AI applications quickly, including standard agent patterns.

LangGraph is a lower-level orchestration framework. It gives you precise control over the workflow, making it suitable for complex, long-running, stateful agents.

A simple comparison:

LangChain	LangGraph
Easier to start	More flexible
Many built-in abstractions	You design the workflow
Good for common agents	Good for complex production agents
Faster development	Fine-grained control

Many developers use **both together**: LangChain provides models, tools, and integrations, while LangGraph manages the overall workflow and state.

In one sentence

LangGraph is a workflow engine for AI agents that lets you build complex, stateful applications by connecting deterministic code and AI-driven decisions into a graph, with support for memory, persistence, human approval, streaming, and production deployment.

One-Sentence Summary

LangGraph is like designing a factory assembly line: each **node** is a worker with one job, the **state** is the shared notebook everyone uses, and the **edges** define how work flows from one worker to the next until the task is complete.